

波浪能装置多自由度响应与 PTO 阻尼优化

摘要

针对波浪能装置中浮子与振子的垂荡响应及 PTO 阻尼优化问题,本文以静水平衡位置为基准,综合考虑附加质量、兴波阻尼、静水恢复力及 PTO 弹簧阻尼作用,建立二自由度耦合动力学模型。针对问题一,将二阶运动方程化为四维一阶状态方程,采用 DOP853 自适应方法对常量阻尼和幂律阻尼两种工况的前 40 个波浪周期进行时域积分,并以 0.2 s 为间隔输出位移与速度。数值收敛、能量闭合和物理适用范围检验均通过,两工况状态量的收敛误差均低于 10^{-7} 。

针对问题二,以周期稳态平均输出功率为统一目标,分别建立常量阻尼的复频域解析优化模型和幂律阻尼的周期射击优化模型。通过全域扫描、参数脊线追踪、多起点精修和边界优化,得到常量阻尼最优系数为 37193.8126 N s/m,幂律阻尼最优参数为 $a = 100000$ 、 $p = 0.415718380$,对应平均功率分别为 229.333940 W 和 229.994333 W。幂律阻尼的最优比例系数取到允许上界,平均功率较常量阻尼提高 0.287961%。零初猜射击、冷启动长时积分和周期能量积分的独立复核结果一致。

针对问题三,在微幅波和小角度条件下建立包含浮子与振子垂荡、纵摇运动的四自由度线性模型,并由封闭薄壳几何与平行轴定理补全物理转动惯量。高精度时域积分得到前 40 个波浪周期的八状态响应,最大垂荡位移为 0.915847 m,最大纵摇角为 0.153758 rad;直线与旋转 PTO 的累计输出分别为 6935.244394 J 和 1.928170 J。严格积分、矩阵指数解和能量闭合验证了结果一致性,壳体口径敏感性说明纵摇峰值规模较稳健,但低阻尼瞬态相位对转动惯量较敏感。

针对问题四,将垂荡与纵摇子系统的频域周期稳态平均功率严格分解为两个单变量函数,并利用 Sherman-Morrison 秩一更新推导解析最优条件。得到直线阻尼最优系数为 59152.916113 N s/m,旋转阻尼最优系数为上边界 100000 N m s,最大总平均功率为 318.679256 W。全域扫描、二维差分进化、可行邻域扰动和周期时域能量积分均验证了该约束全局最优解。

关键字: 波浪能 耦合动力学 垂荡与纵摇 PTO 阻尼 参数优化

一、问题重述

1.1 问题背景

波浪能装置通过浮子与内部振子的相对运动驱动能量输出系统（power take-off, PTO）中的阻尼器做功，将波浪的机械能转化为可利用能量。浮子在微幅周期波中运动时，除受波浪激励外，还会受到附加惯性力、兴波阻尼力和静水恢复力。因此，准确描述浮子、振子与 PTO 之间的耦合关系，是后续优化输出功率的基础。

1.2 问题一的要求

问题一只考虑浮子和振子的竖直垂荡运动。已知波浪激励力为 $f \cos(\omega t)$ ，角频率取 $\omega = 1.4005 \text{ s}^{-1}$ ，角度均采用弧度制。需要分别对常量阻尼系数 10000 N s/m 和阻尼系数随相对速度绝对值的 0.5 次幂变化两种工况建模，计算前 40 个波浪周期内的位移和速度，并给出 10、20、40、60 和 100 s 时的指定结果。

1.3 问题二的要求

问题二沿用垂荡运动模型，改用 $\omega = 2.2143 \text{ s}^{-1}$ 的水动力参数。需分别在常量阻尼系数 $c \in [0, 100000]$ 以及幂律阻尼参数 $a \in [0, 100000]$ 、 $p \in [0, 1]$ 的约束下，确定使 PTO 平均输出功率最大的阻尼参数与最大功率。

1.4 问题三的要求

问题三允许浮子同时作垂荡和纵摇运动，振子随中轴转动并沿中轴滑动。在 $\omega = 1.7152 \text{ s}^{-1}$ 、直线阻尼系数 10000 N s/m 和旋转阻尼系数 1000 N m s 下，需建立垂荡-纵摇动力学模型，计算前 40 个波浪周期内的垂荡位移与速度、纵摇角与角速度，并按 0.2 s 间隔输出正式结果。

1.5 问题四的要求

问题四沿用垂荡-纵摇四自由度模型，采用 $\omega = 1.9806 \text{ s}^{-1}$ 的水动力参数。直线与旋转 PTO 阻尼系数均为常量，并分别在 $[0, 100000]$ 内取值，需求使两类 PTO 总平均输出功率最大的两个阻尼系数及最大功率。

二、问题分析

2.1 问题一的分析

问题一属于外周期激励下的二自由度耗散振动问题。模型的关键有三点：一是将附加质量并入浮子的有效惯性；二是根据水线面几何确定静水恢复刚度；三是保证弹簧力与 PTO 阻尼力作为内力，在浮子和振子方程中符号相反。常量阻尼工况为线性系统，幂律阻尼工况为非线性系统，两者可在统一状态方程下进行求解和对比。

2.2 问题二的分析

问题二的核心是在同一周期稳态口径下极大化 PTO 平均耗散功率。常量阻尼对应线性定常系统，可利用复频域稳态解将优化化为一维问题；幂律阻尼则使系统呈现速度相关非线性，需直接求解周期轨道并在二维约束域内搜索。为避免初始自由振动影响目标值，本文不使用从零初态起算的全程平均，而使用单个稳定周期的平均功率。

2.3 问题三的分析

问题三将运动扩展为浮子和振子的垂荡与纵摇四自由度响应。在微幅波和小角度一阶线性化下，垂荡与纵摇之间的交叉项为二阶小量，因此整体模型可分解为垂荡和纵摇两个耦合二自由度子系统。建模难点在于由壳体几何补全物理转动惯量，并保留振子绕移动铰轴运动所导致的纵摇非对角惯性耦合。

2.4 问题四的分析

问题四中两类 PTO 阻尼均为常量，因此可直接在频域求周期稳态响应。由于一阶模型的垂荡与纵摇子系统块对角分离，总功率是两个各依赖一个阻尼系数的功率函数之和，二维联合优化可严格分解为两个一维有界优化。在此基础上，可由秩一阻尼更新推导解析最优点，再以全域扫描、二维随机优化和时域积分交叉验证。

三、模型假设

1. 问题一和问题二仅考虑浮子与振子的竖直垂荡；问题三和问题四进一步考虑侧视平面内的纵摇。
2. 海水无黏、无旋，附加质量和兴波阻尼采用题目给定的线性系数。
3. 位移相对静水平衡位置定义，因此重力、静浮力和弹簧的静态预压缩量在增量方程中相互抵消。
4. 浮子运动过程中水线始终位于圆柱壳体段，水线面积可取 $A_w = \pi R^2$ 。
5. 忽略中轴、底座、隔层和 PTO 自身的质量，忽略题目未给出的其他摩擦。

6. 问题三和问题四满足微幅波和小角度条件，只保留位移、转角及其导数的一阶项。
7. 主模型将浮子视为密度均匀的封闭薄壳，计入圆柱侧面、圆锥侧面和顶部圆盖。

四、符号说明

表 1 主要符号

符号	含义	单位
x_f, x_o	浮子、振子相对静水平衡位置的垂荡位移	m
v_f, v_o	浮子、振子的垂荡速度	m/s
r, v_r	振子相对浮子的位移和速度	m, m/s
m_f, m_o, m_a	浮子质量、振子质量和附加质量	kg
b, k, k_h	兴波阻尼系数、弹簧刚度和静水恢复刚度	N s/m, N/m
$D(v_r)$	PTO 阻尼力函数	N
f, ω	波浪激励力幅值和角频率	N, s ⁻¹
c	常量阻尼系数	N s/m
a, p	幂律阻尼的比例系数和幂指数	随 p 变化，无量纲
$\bar{P}$	周期稳态平均 PTO 输出功率	W
θ_f, θ_o	浮子和振子的绝对纵摇角	rad
$J_{f,C}, J_{o,C}$	浮子、振子绕自身质心横轴的转动惯量	kg m ²
$J_{o,P}$	振子绕下部铰轴的等效转动惯量	kg m ²
c_z, c_θ	直线与旋转 PTO 阻尼系数	N s/m, N m s

五、问题一模型的建立与求解

5.1 坐标选取与参数确定

以浮子和振子的静水平衡位置为坐标原点，规定竖直向上为正方向。记

$$r = x_o - x_f, \quad v_r = v_o - v_f. \quad (1)$$

附件给出 $m_f = 4866$ kg、 $m_o = 2433$ kg 和 $m_a = 1335.535$ kg，故浮子的垂荡有效质量为

$$M_f = m_f + m_a = 6201.535 \text{ kg}. \quad (2)$$

根据浮子与振子的总质量反算排水体积，所得静态吃水为 2.000010 m，水线位于圆柱段的 0–3 m 范围内。因此水线面积为 πR^2 ，静水恢复刚度为

$$k_h = \rho g \pi R^2 = 31557.298205 \text{ N/m}. \quad (3)$$

题目给定 $b = 656.3616 \text{ N s/m}$ 、 $f = 6250 \text{ N}$ 、 $k = 80000 \text{ N/m}$ ，由 $\omega = 1.4005 \text{ s}^{-1}$ 得波浪周期

$$T = \frac{2\pi}{\omega} = 4.486387224 \text{ s}. \quad (4)$$

5.2 浮子-振子耦合动力学方程

波浪激励力只直接作用于浮子。弹簧力和 PTO 阻尼力是浮子与振子之间的内力，在两个物体的动力学方程中大小相等、方向相反。由牛顿第二定律得

$$M_f \ddot{x}_f = f \cos(\omega t) - b\dot{x}_f - k_h x_f + k(x_o - x_f) + D(\dot{x}_o - \dot{x}_f), \quad (5)$$

$$m_o \ddot{x}_o = -k(x_o - x_f) - D(\dot{x}_o - \dot{x}_f). \quad (6)$$

初始时刻两者均静止于静水平衡位置，因而

$$x_f(0) = x_o(0) = \dot{x}_f(0) = \dot{x}_o(0) = 0. \quad (7)$$

5.3 阻尼规律与物理约束

两种工况的 PTO 阻尼力分别写为

$$D_1(v_r) = 10000 v_r, \quad (8)$$

$$D_2(v_r) = 10000 |v_r|^{0.5} v_r. \quad (9)$$

两种情况均满足 $D(v_r)v_r \geq 0$ ，即阻尼器的瞬时耗散功率非负，不会向机械系统注入能量。

5.4 一阶状态方程与数值方法

取状态向量 $\mathbf{y} = [x_f, v_f, x_o, v_o]^T$ ，将式 (5)–(6) 改写为

$$\begin{cases} \dot{x}_f = v_f, \\ \dot{v}_f = \frac{f \cos(\omega t) - b v_f - k_h x_f + k r + D(v_r)}{M_f}, \\ \dot{x}_o = v_o, \\ \dot{v}_o = \frac{-k r - D(v_r)}{m_o}. \end{cases} \quad (10)$$

本文采用 DOP853 高阶自适应 Runge–Kutta 方法积分式 (10)，相对容差取 10^{-10} ，绝对容差取 10^{-12} ，最大内部步长取 0.02 s 。计算时间覆盖 $40T = 179.455489 \text{ s}$ ，并在 $0\text{--}179.4 \text{ s}$

上按 0.2 s 间隔输出，共得 898 个时刻。题目指定的五个时刻均位于输出网格上，可直接取值，无需额外插值。

5.5 求解结果与响应特征

两种阻尼规律下的位移响应如图 1 所示。在初始阶段，系统的自由振动与波浪强迫振动叠加，形成明显的振幅包络；随着兴波阻尼和 PTO 阻尼不断耗散机械能，自由振动分量逐渐衰减，系统趋于以入射波浪频率为主的周期响应。

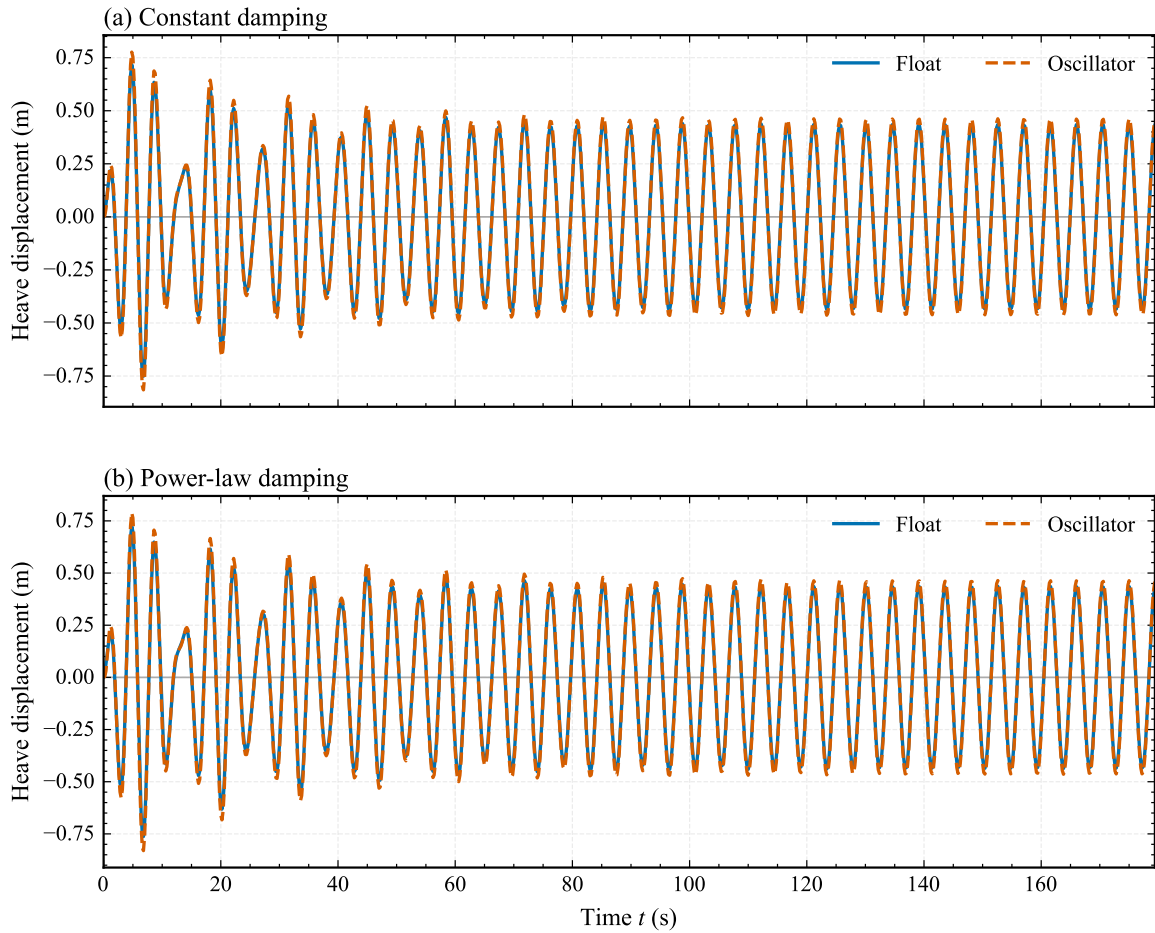


图 1 两种阻尼规律下浮子与振子的垂荡位移响应

速度响应见图 2。振子与浮子的运动趋势总体一致，但始终存在小幅的振幅差与相位差，由此产生 PTO 阻尼器做功所需的相对速度。

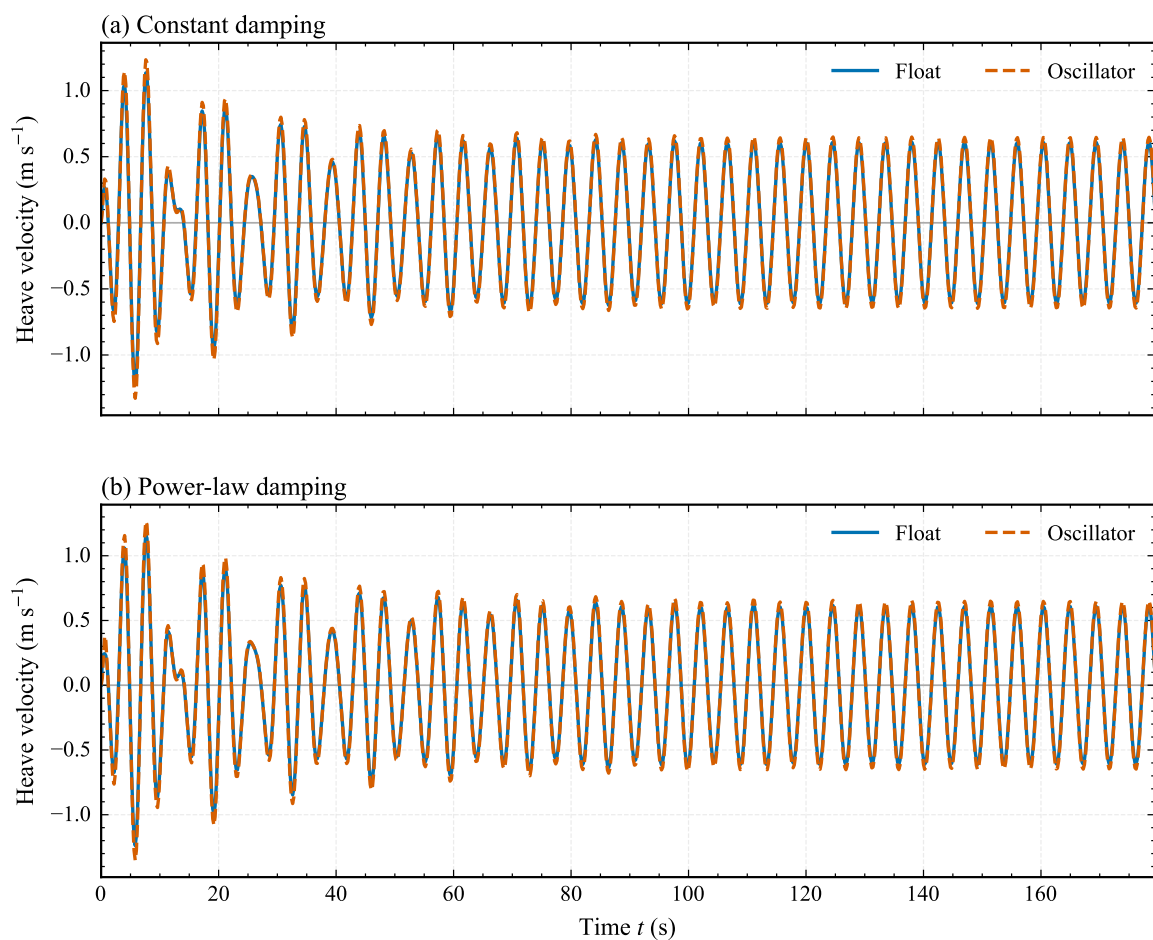


图 2 两种阻尼规律下浮子与振子的垂荡速度响应

常量阻尼工况在题目指定时刻的计算结果如表 2 所示，幂律阻尼的相应结果如表 3 所示。

表 2 常量阻尼工况的指定时刻响应

时间/s	浮子位移/m	浮子速度/(m/s)	振子位移/m	振子速度/(m/s)
10	-0.190711	-0.641009	-0.211679	-0.693953
20	-0.590684	-0.240951	-0.634248	-0.272776
40	0.285374	0.312971	0.296499	0.332912
60	-0.314506	-0.479455	-0.331436	-0.515728
100	-0.083615	-0.604211	-0.084068	-0.643002

表 3 幂律阻尼工况的指定时刻响应

时间/s	浮子位移/m	浮子速度/(m/s)	振子位移/m	振子速度/(m/s)
10	-0.205877	-0.652821	-0.234572	-0.699937
20	-0.611108	-0.254784	-0.661061	-0.277021
40	0.268768	0.295302	0.280157	0.312523
60	-0.327164	-0.491518	-0.349606	-0.525587
100	-0.088408	-0.609831	-0.093493	-0.650076

图 3 展示了第 38–40 周期的相对运动。在当前相对速度范围内，幂律阻尼的等效阻尼系数小于常量阻尼系数，因而其耗散作用相对较弱，相对位移和相对速度均略大。第 40 周期内，常量阻尼下浮子和振子的位移半峰峰值分别为 0.435354 m 和 0.462033 m；幂律阻尼下分别为 0.435983 m 和 0.463784 m。

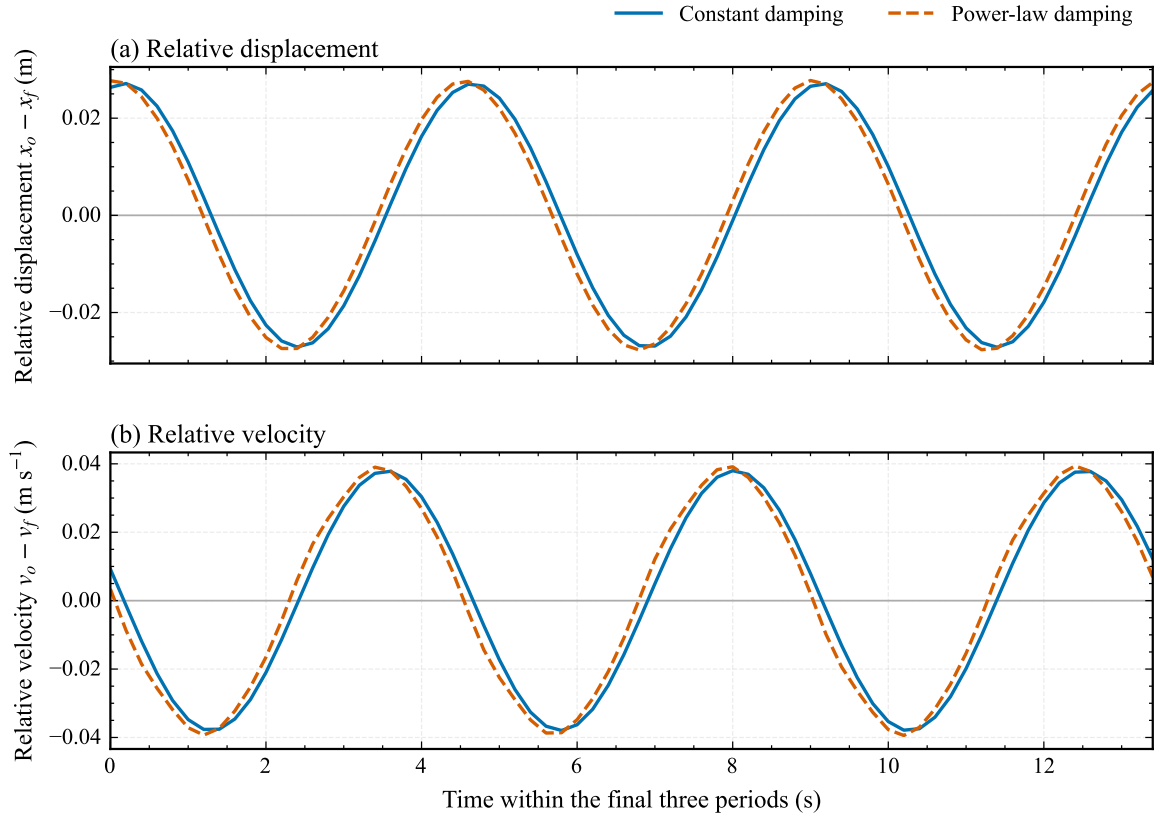


图 3 第 38–40 周期内两种阻尼规律的相对运动对比

5.6 模型检验

为检验数值结果的可靠性，本文从数值收敛、能量闭合和物理适用范围三个方面进行验证。

1. 数值收敛性：将最大内部步长由 0.02 s 缩小至 0.01 s，并进一步收紧积分容差。常量阻尼和幂律阻尼工况四个状态量的最大差分别为 1.439×10^{-13} 和 2.422×10^{-8} ，

均低于 10^{-7} 。

2. 能量闭合：根据波浪输入功、机械能变化、兴波耗散和 PTO 耗散构建能量平衡式。两工况的最大相对能量残差分别为 6.841×10^{-15} 和 2.320×10^{-13} ，说明方程符号和数值积分保持了能量一致性。
3. 物理范围：计算过程中圆柱段吃水始终处于 1.276–2.763 m，未超出圆柱段；弹簧长度始终为正。因此线性静水恢复力与结构几何假设在所得响应范围内自洽。

此外，常量阻尼时域解与复频域解析稳态解的最大相对误差为 2.952×10^{-14} ，进一步从独立求解路径验证了线性工况的正确性。

5.7 问题一小结

本文建立的二自由度垂荡耦合模型同时包含浮子有效惯性、兴波阻尼、静水恢复力以及浮子–振子之间的弹簧和 PTO 阻尼耦合。计算得到两种阻尼规律下前 40 个波浪周期的位移和速度序列，并完成了题目指定时刻的结果输出。幂律阻尼工况在当前速度范围内的相对运动略大，这一结论为问题二中以平均输出功率为目标的阻尼参数优化提供了基础。

六、 问题二 PTO 阻尼参数优化

6.1 优化目标与稳态口径

问题二采用附件 3 的情形 2，其水动力参数为

$$\omega = 2.2143 \text{ s}^{-1}, \quad m_a = 1165.992 \text{ kg}, \quad b = 167.8395 \text{ N s/m}, \quad f = 4890 \text{ N}. \quad (11)$$

此时浮子有效质量为 $M_f = 6031.992 \text{ kg}$ ，波浪周期为 $T = 2.837549251 \text{ s}$ 。其余结构参数与动力学方程沿用问题一。

由于本问评价装置在周期波浪下的持续输出能力，本文将目标定义为系统进入周期稳态后一个完整波浪周期内的平均 PTO 输出功率。若 $\mathbf{y}_s(t+T) = \mathbf{y}_s(t)$ ，则

$$\bar{P} = \frac{1}{T} \int_{t_s}^{t_s+T} P_{\text{PTO}}(t) dt. \quad (12)$$

该口径排除了初始自由振动对优化结果的影响，使不同阻尼参数的目标值可直接比较。

6.2 常量阻尼的复频域优化

当 PTO 阻尼系数为常量 c 时，系统为线性定常系统。定义

$$\mathbf{M} = \begin{bmatrix} 6031.992 & 0 \\ 0 & 2433 \end{bmatrix}, \quad \mathbf{C}(c) = \begin{bmatrix} 167.8395 + c & -c \\ -c & c \end{bmatrix}, \quad (13)$$

$$\mathbf{K} = \begin{bmatrix} 31557.298205 + 80000 & -80000 \\ -80000 & 80000 \end{bmatrix}. \quad (14)$$

设周期稳态位移为 $\mathbf{x}(t) = \Re\{\mathbf{X}e^{i\omega t}\}$ ，则复振幅满足

$$(\mathbf{K} - \omega^2 \mathbf{M} + i\omega \mathbf{C}(c)) \mathbf{X} = \begin{bmatrix} 4890 \\ 0 \end{bmatrix}. \quad (15)$$

因此常量阻尼的目标函数为

$$\bar{P}(c) = \frac{1}{2} c \omega^2 |[-1, 1] \mathbf{X}(c)|^2, \quad 0 \leq c \leq 100000. \quad (16)$$

首先对完整可行区间进行 10001 点扫描，确认目标函数只有一个内部峰值；再采用有界一维优化精修。全区间扫描、数值优化与秩一阻尼更新导出的解析最优条件相互一致，最终得到

$$\boxed{c^* = 37193.8126 \text{ N s/m}}, \quad \boxed{\bar{P}_{\max} = 229.333940 \text{ W}}. \quad (17)$$

6.3 幂律阻尼的周期射击优化

对幂律阻尼，记比例系数为 a 、幂指数为 p ，则阻尼力与瞬时功率为

$$D(v_r) = a|v_r|^p v_r, \quad P_{\text{PTO}} = a|v_r|^{p+2} \geq 0, \quad (18)$$

其中 $0 \leq a \leq 100000$ 、 $0 \leq p \leq 1$ 。比例系数 a 的量纲随 p 变化，因此不将其统一标注为常量阻尼系数的单位。

为直接定位稳定周期轨道，定义状态经过一个周期后的映射 Φ_T ，并求解射击条件

$$\Phi_T(\mathbf{y}_0; a, p) - \mathbf{y}_0 = \mathbf{0}. \quad (19)$$

得到周期初态后，将式 (18) 中的瞬时功率作为附加状态积分，便可计算目标函数

$$\bar{P}(a, p) = \frac{1}{T} \int_0^T a|v_r(t; a, p)|^{p+2} dt. \quad (20)$$

优化采用“全域粗网格-脊线追踪-多起点内部精修-边界精修-最终严格复核”的分层路线。21 × 11 粗网格表明，高功率区域形成从低指数、低系数向高指数、高系数延伸的参数补偿脊线，如图 4 所示。橙色虚线为在各固定指数下优化比例系数得到的高功率脊线，星标为最终解。

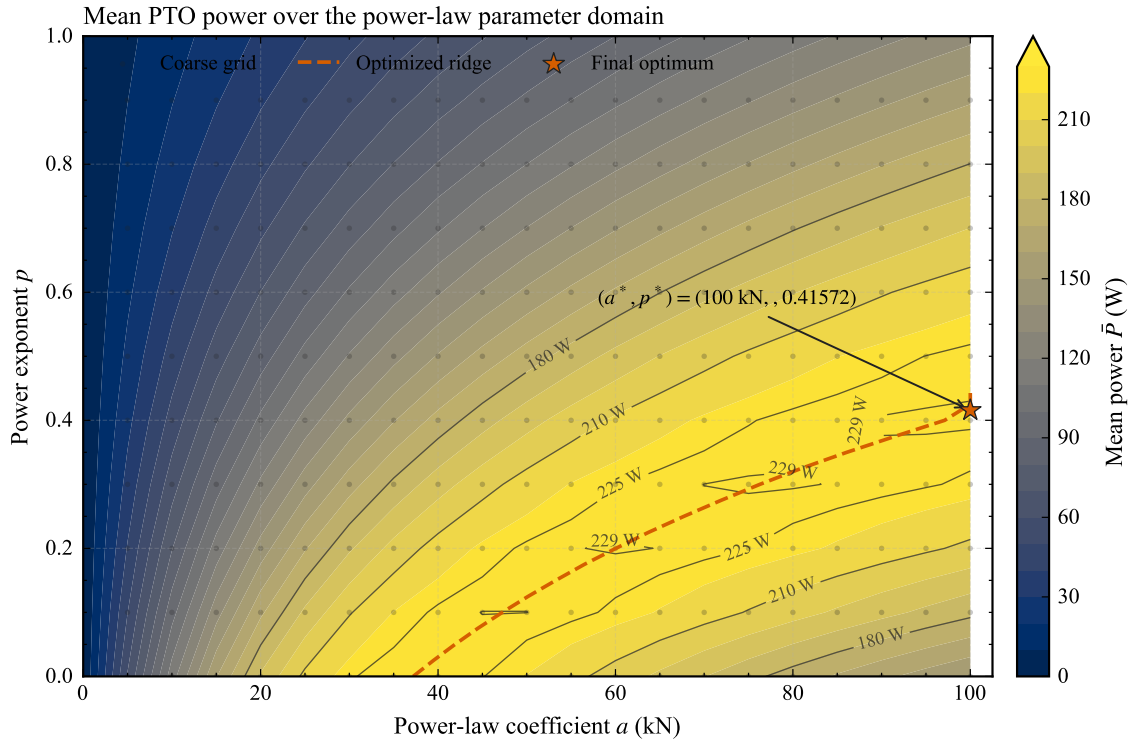


图 4 幂律阻尼参数域内的周期稳态平均功率与高功率脊线

脊线随 p 增大持续向 a 的上边界移动，多个内部初值也均指向同一边界。因此固定真实边界 $a = 100000$ ，对 $p \in [0, 1]$ 作 501 点扫描并严格精修，最终得到

$$\boxed{a^* = 100000}, \quad \boxed{p^* = 0.415718380}, \quad \boxed{\bar{P}_{\max} = 229.994333 \text{ W}}. \quad (21)$$

最优比例系数取到允许上界，而幂指数位于区间内部。在最优指数两侧作微小扰动，功率均下降；固定最优指数并将 a 从上边界向可行域内移动，功率同样下降，说明 $a = 100000$ 为活跃约束。

两类阻尼的周期稳态平均功率优化曲线如图 5 所示。常量阻尼在完整区间内只有一个内部最大值；幂律阻尼在 $a = 100000$ 时，指数方向也只有一个内部峰值。

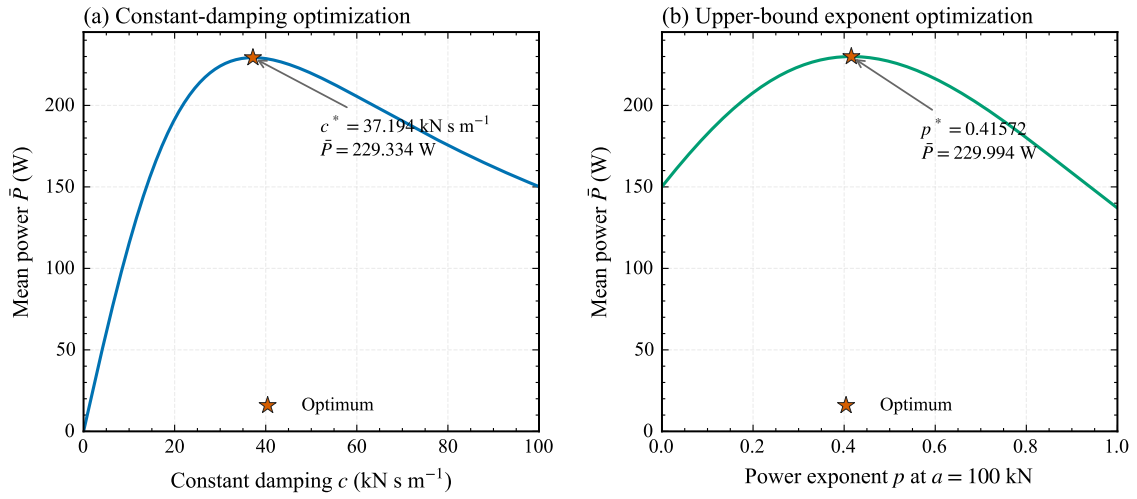


图 5 两类 PTO 阻尼的周期稳态平均功率优化曲线

6.4 最优结果比较

两种阻尼规律的最优参数和最大平均功率汇总于表 4。

表 4 问题二的最优阻尼参数与最大平均功率

阻尼形式	最优参数	最大平均功率/W
常量阻尼	$c^* = 37193.8126 \text{ N s/m}$	229.333940
幂律阻尼	$a^* = 100000, p^* = 0.415718380$	229.994333

幂律阻尼的最大平均功率比常量阻尼提高

$$229.994333 - 229.333940 = 0.660393 \text{ W}, \quad (22)$$

相对增幅为 0.287961%。两种最优阻尼下一个稳定周期内的相对位移、相对速度和瞬时 PTO 功率如图 6 所示。图中归一化时间为 t/T 。

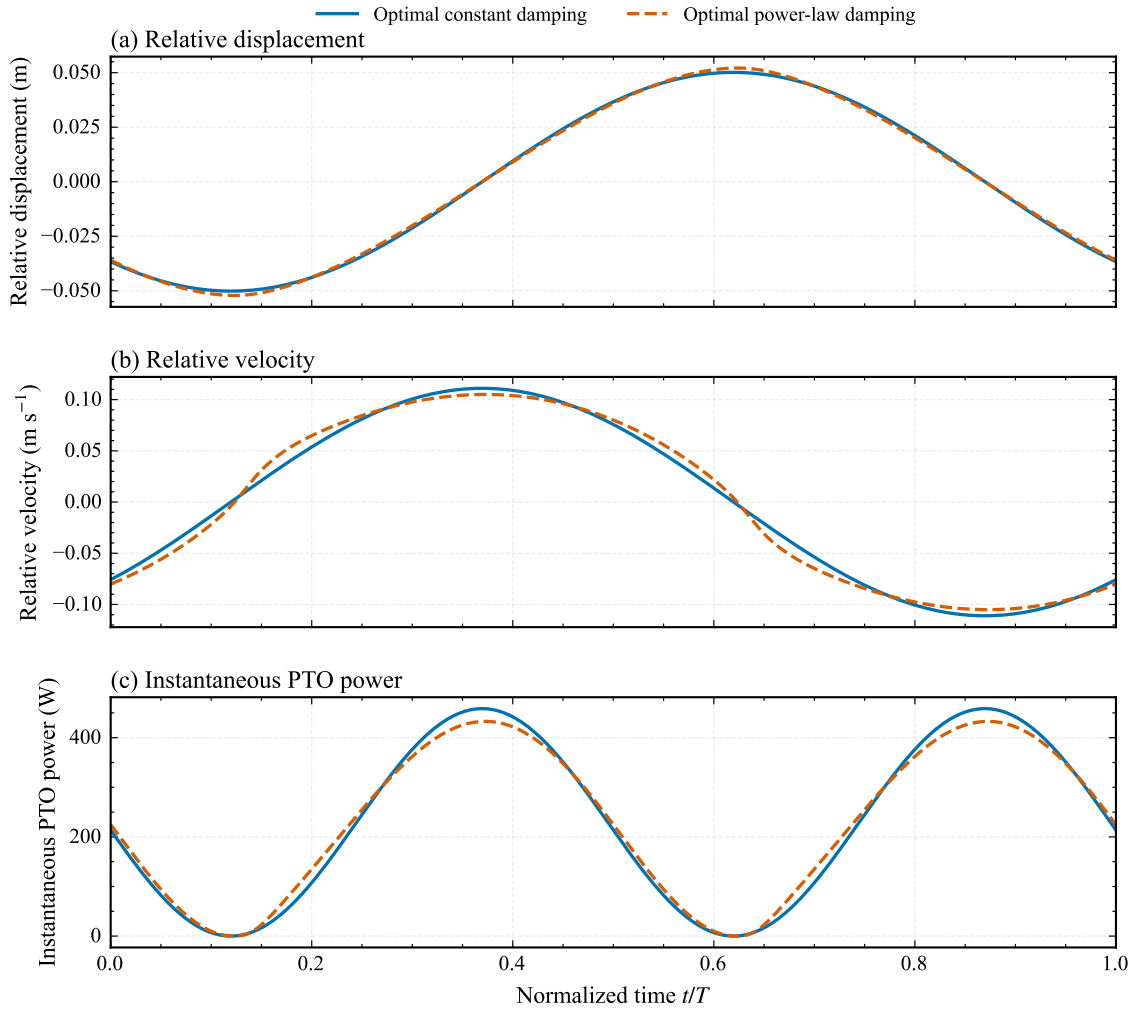


图 6 两种最优 PTO 阻尼下一个稳定周期的响应与输出功率

幂律阻尼最优周期内的关键响应与物理范围汇总于表 5。

表 5 幂律阻尼最优周期的关键指标

指标	数值
单周期 PTO 耗散能量	652.620247 J
瞬时 PTO 功率最大值	432.909259 W
最大相对位移	0.052148 m
最大相对速度	0.105093 m/s
圆柱段吃水范围	1.550772–2.449248 m
弹簧长度范围	0.149810–0.254105 m

6.5 优化结果检验

常量阻尼最优点两侧扰动均使功率下降，复频域动力平衡残差为 8.13×10^{-12} N，时域周期解与频域功率的相对误差为 1.99×10^{-10} 。对幂律阻尼最优解，进一步使用三条不同的路径复核：

1. 零初猜射击：从全零周期初态启动射击，所得功率与最终严格射击结果的相对差为 2.57×10^{-10} ，周期闭合误差为 6.78×10^{-12} 。
2. 冷启动长时积分：从静水平衡零初态出发，经过 239 个周期进入稳定周期，平均功率与严格射击结果的相对差为 1.28×10^{-8} 。
3. 周期能量积分：在最优周期内取 8001 个采样点，直接积分瞬时功率。所得平均功率为 229.994332893 W，与严格射击结果的相对差为 1.42×10^{-10} 。

最优周期内 PTO 瞬时功率始终非负，吃水和弹簧长度也均满足模型的物理适用范围，说明优化结果同时满足周期稳态、能量耗散和结构几何约束。

七、问题三垂荡-纵摇四自由度模型

7.1 坐标与一阶线性化

以系统的静水平衡位置为原点，规定竖直向上为垂荡正方向，侧视平面内逆时针为纵摇正方向。记浮子和振子的垂荡位移为 x_f, x_o ，绝对纵摇角为 θ_f, θ_o 。直线 PTO 的相对位移和速度分别为 $x_o - x_f$ 和 $\dot{x}_o - \dot{x}_f$ ，旋转 PTO 的相对转角和角速度分别为 $\theta_o - \theta_f$ 和 $\dot{\theta}_o - \dot{\theta}_f$ 。

根据振子质心的平面几何关系，在微幅波和小角度条件下，垂荡与纵摇之间的交叉项为二阶小量。因此一阶模型可写为垂荡和纵摇两个块对角子系统，但纵摇子系统内部的非对角惯性项仍需保留。

7.2 壳体几何与转动惯量

附件仅给出浮子的纵摇附加转动惯量，物理转动惯量需结合实际转轴补全。主模型将浮子视为密度均匀的封闭薄壳，按圆柱侧面、圆锥侧面和顶部圆盖的面积分配总质量，再应用平行轴定理。得浮子质心到下部铰轴的距离和绕质心的转动惯量为

$$a_f = 1.407925157 \text{ m}, \quad J_{f,C} = 8398.776060 \text{ kg m}^2. \quad (23)$$

弹簧的静平衡长度为

$$l_e = l_0 - \frac{m_o g}{k} = 0.2019575 \text{ m}, \quad (24)$$

故振子质心到铰轴的距离 $d = l_e + h_o/2 = 0.4519575 \text{ m}$ 。圆柱振子绕自身质心横轴的转动惯量为 $J_{o,C} = 202.75 \text{ kg m}^2$ ，由平行轴定理得其绕铰轴的等效转动惯量

$$J_{o,P} = J_{o,C} + m_o d^2 = 699.728161 \text{ kg m}^2. \quad (25)$$

7.3 四自由度动力学方程

令 $\mathbf{q}_z = [x_f, x_o]^T$ 、 $\mathbf{q}_\theta = [\theta_f, \theta_o]^T$ 。问题三的波浪角频率为 $\omega = 1.7152 \text{ s}^{-1}$ 。垂荡子系统满足

$$\mathbf{M}_z \ddot{\mathbf{q}}_z + \mathbf{C}_z \dot{\mathbf{q}}_z + \mathbf{K}_z \mathbf{q}_z = \begin{bmatrix} 3640 \cos(1.7152t) \\ 0 \end{bmatrix}, \quad (26)$$

其中

$$\mathbf{M}_z = \begin{bmatrix} 5894.876 & 0 \\ 0 & 2433 \end{bmatrix}, \quad \mathbf{C}_z = \begin{bmatrix} 10683.4558 & -10000 \\ -10000 & 10000 \end{bmatrix}, \quad (27)$$

$$\mathbf{K}_z = \begin{bmatrix} 111557.298205 & -80000 \\ -80000 & 80000 \end{bmatrix}. \quad (28)$$

纵摇子系统满足

$$\mathbf{M}_\theta \ddot{\mathbf{q}}_\theta + \mathbf{C}_\theta \dot{\mathbf{q}}_\theta + \mathbf{K}_\theta \mathbf{q}_\theta = \begin{bmatrix} 1690 \cos(1.7152t) \\ 0 \end{bmatrix}, \quad (29)$$

其中

$$\mathbf{M}_\theta = \begin{bmatrix} 20223.512213 & -1548.172239 \\ -1548.172239 & 699.728161 \end{bmatrix}, \quad (30)$$

$$\mathbf{C}_\theta = \begin{bmatrix} 1654.3383 & -1000 \\ -1000 & 1000 \end{bmatrix}, \quad \mathbf{K}_\theta = \begin{bmatrix} 292460.422695 & -250000 \\ -250000 & 239223.796545 \end{bmatrix}. \quad (31)$$

式 (30) 中的非对角项对应移动铰轴对浮子和振子纵摇加速度的惯性耦合，不能在四自由度模型中略去。

7.4 数值方法与正式输出网格

初始时刻的四个位移与四个速度状态均为零。采用 DOP853 自适应积分方法，相对容差取 10^{-10} ，最大内部步长取 0.01 s 。波浪周期为 $T = 3.663237702 \text{ s}$ ，积分至精确的第 40 周期终点 $40T = 146.529508 \text{ s}$ 。正式结果在 $0-146.4 \text{ s}$ 上按 0.2 s 间隔输出，共 733 个时刻。其中 0.2 s 是题目要求的输出间隔，不是积分器的内部步长。

7.5 响应结果与指定时刻数值

浮子与振子的垂荡位移和速度如图 7 所示，纵摇角与角速度如图 8 所示。两类运动在初始阶段均包含明显的自由振动包络，随后逐渐转为以波浪频率为主的受迫响应。浮子与振子的同类状态总体接近，但仍保留了产生 PTO 输出所需的相对速度和相对角速度。

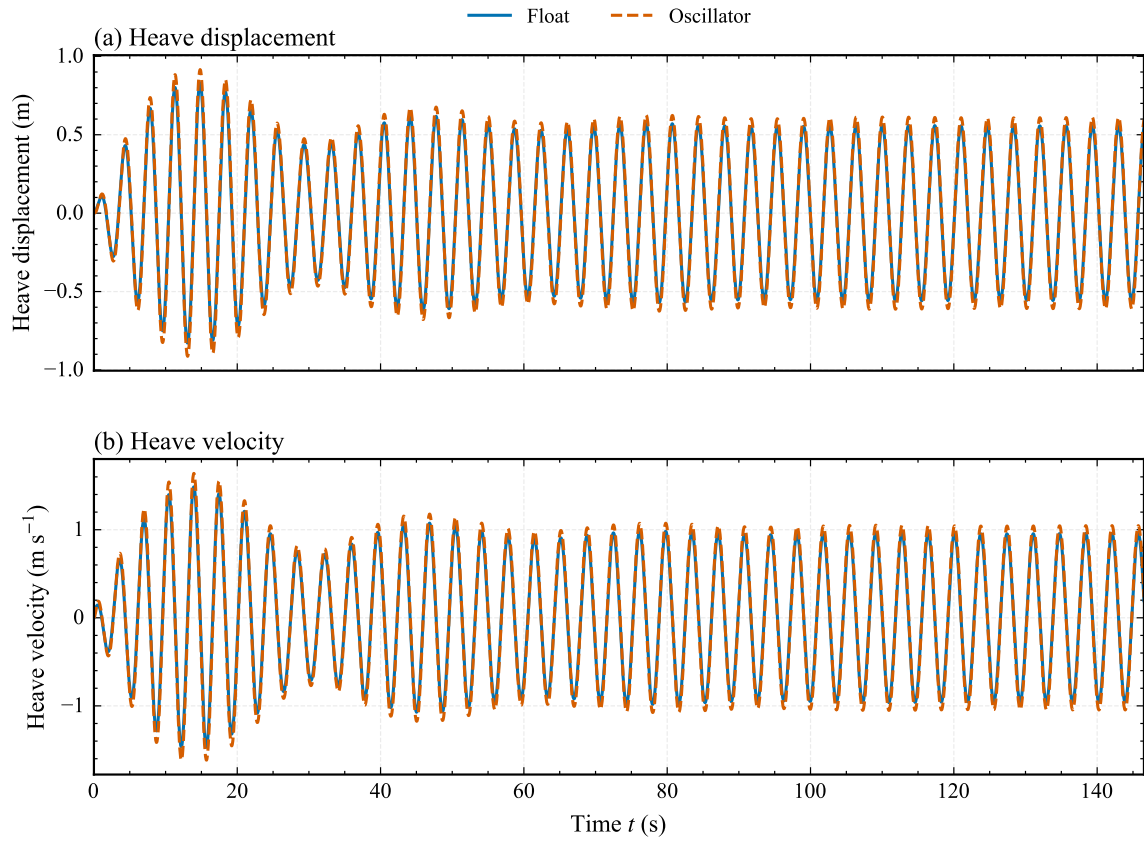


图 7 情形 3 下浮子与振子的垂荡位移和速度响应

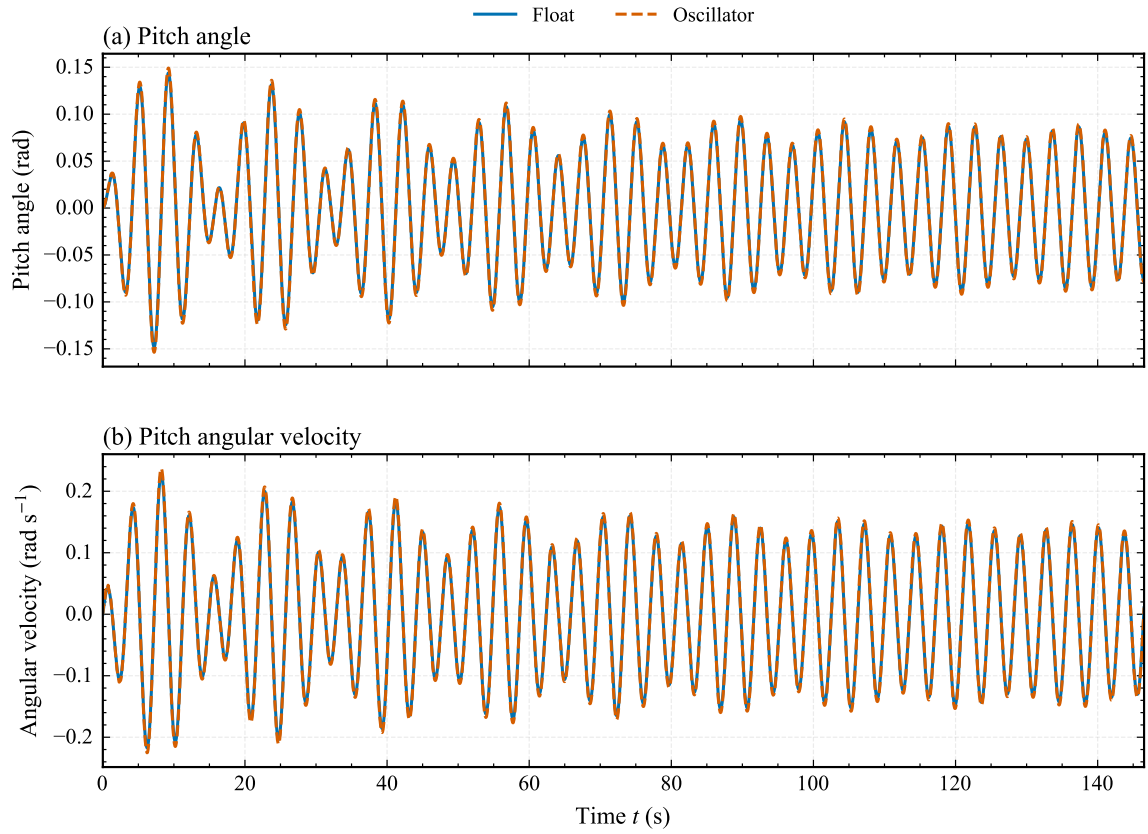


图 8 情形 3 下浮子与振子的纵摇角和角速度响应

题目指定的 10、20、40、60 和 100 s 时刻的垂荡结果见表 6，纵摇结果见表 7。

表 6 问题三指定时刻的垂荡响应

时间/s	浮子位移/m	浮子速度/(m/s)	振子位移/m	振子速度/(m/s)
10	-0.528276	0.969851	-0.598729	1.038201
20	-0.704815	-0.269306	-0.772294	-0.319026
40	0.369361	0.757580	0.392653	0.844962
60	-0.320675	-0.721762	-0.341426	-0.799321
100	-0.050154	-0.946690	-0.042612	-1.036526

表 7 问题三指定时刻的纵摇响应

时间/s	浮子角/rad	浮子角速度/(rad/s)	振子角/rad	振子角速度/(rad/s)
10	0.054994	-0.197808	0.057153	-0.204933
20	0.086335	-0.045551	0.089398	-0.047403
40	-0.108234	-0.076354	-0.112148	-0.079135
60	0.047511	0.119844	0.049093	0.124173
100	0.038079	0.118554	0.039434	0.122668

全过程浮子和振子的最大垂荡位移绝对值分别为 0.831274 m 和 0.915847 m，最大

纵摇角绝对值分别为 0.148295 rad 和 0.153758 rad。最大纵摇角约为 8.81° ，未超过本文设定的 0.2 rad 小角度预警线。

7.6 PTO 输出与能量特征

直线与旋转 PTO 的瞬时输出功率分别为

$$P_z = 10000(\dot{x}_o - \dot{x}_f)^2 \geq 0, \quad P_\theta = 1000(\dot{\theta}_o - \dot{\theta}_f)^2 \geq 0. \quad (32)$$

至精确第 40 周期终点，直线 PTO 和旋转 PTO 的累计输出能量分别为 6935.244394 J 和 1.928170 J，瞬时功率最大值分别为 245.428736 W 和 0.069767 W。旋转 PTO 输出比直线 PTO 小三个数量级以上，说明在题给阻尼参数下，装置能量输出主要来自垂荡方向的直线 PTO。

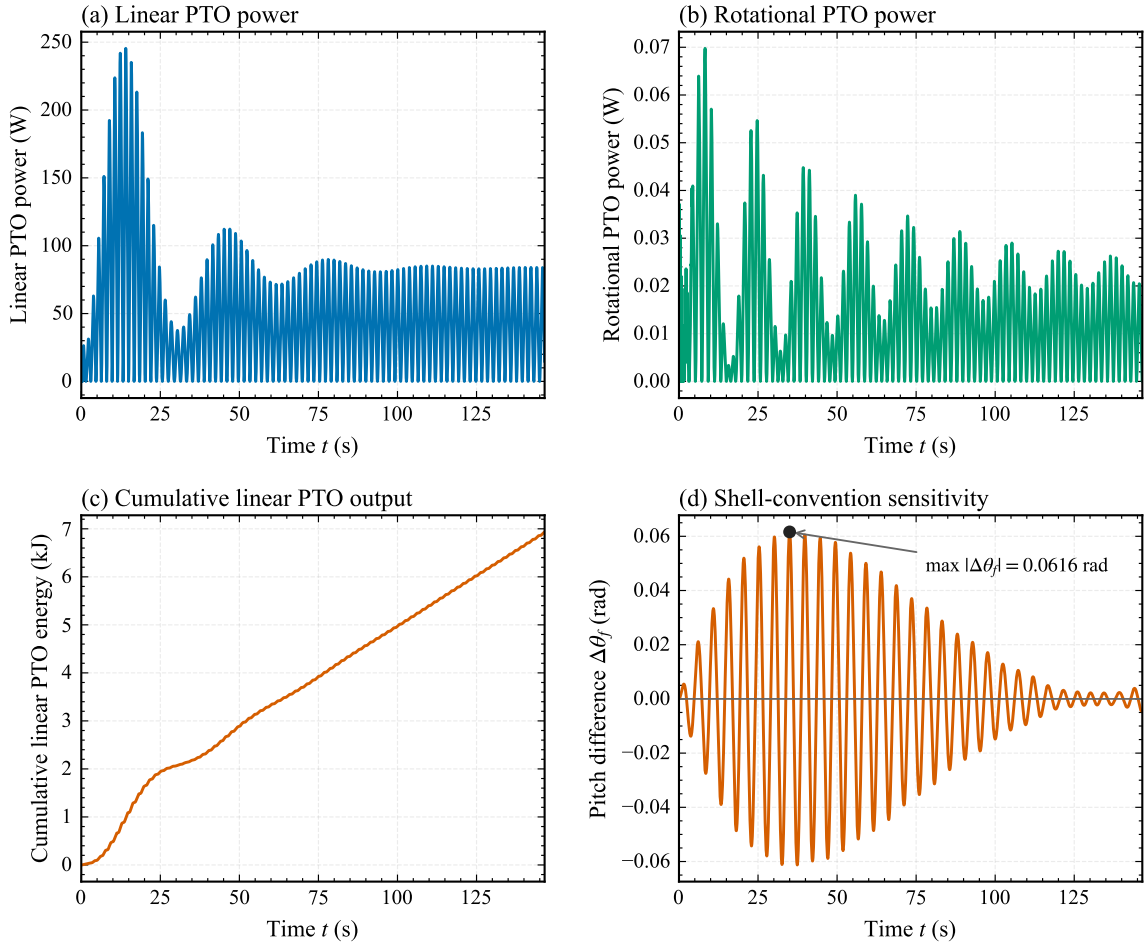


图 9 两类 PTO 输出与浮子壳体口径敏感性

7.7 数值验证与模型敏感性

问题三的结果采用三条数值路线和能量恒等式交叉验证。主积分与收紧容差和步长的参考积分之间，全局最大归一化差为 1.24×10^{-13} ；与常系数线性系统的矩阵指数解之间，最大归一化差为 6.51×10^{-14} 。将波浪输入功、机械能变化、垂荡与纵摇兴波耗散以及两类 PTO 输出纳入能量恒等式，精确 $40T$ 终点的相对残差为 1.12×10^{-14} 。

全过程弹簧长度保持在 $0.114923\text{--}0.288675\text{ m}$ ，始终为正；最大纵摇角也低于小角度预警线，因此结构几何和一阶线性化在主响应范围内自治。

由于题面未明确薄壳是否包含顶部圆盖，另以“不计顶部圆盖”的侧壳口径进行敏感性计算。图 9(d) 中 $\Delta\theta_f$ 表示“替代口径减主口径”。两种口径下四个垂荡状态完全不变，纵摇绝对峰值变化约 1%–2%；但由于低阻尼模态存在瞬态相位累积，浮子和振子纵摇角的最大逐时差分别达到 0.061615 rad 和 0.064300 rad 。因此纵摇峰值规模对壳体口径较稳健，而低阻尼纵摇瞬态的相位对转动惯量口径较敏感。主结果始终采用计入顶部圆盖的封闭薄壳口径，替代口径不混入正式结果表。

八、问题四直线与旋转 PTO 阻尼联合优化

8.1 情形 4 参数与频域目标

问题四采用附件 3 的情形 4，水动力参数为

$$\omega = 1.9806\text{ s}^{-1}, \quad m_a = 1091.099\text{ kg}, \quad J_a = 7142.493\text{ kg m}^2, \quad (33)$$

$$b_z = 528.5018\text{ N s/m}, \quad b_\theta = 1655.909\text{ N m s}, \quad F = 1760\text{ N}, \quad L = 2140\text{ N m}. \quad (34)$$

沿用问题三的封闭薄壳口径和四自由度一阶模型，仅将水动力参数替换为情形 4。记直线和旋转 PTO 阻尼系数为 c_z, c_θ ，二者均满足

$$0 \leq c_z \leq 100000, \quad 0 \leq c_\theta \leq 100000. \quad (35)$$

对 $j \in \{z, \theta\}$ ，令 $\mathbf{r} = [1, -1]^\top$ ，设周期稳态位移为 $\mathbf{q}_j(t) = \Re\{\mathbf{X}_j \mathbf{e}^{i\omega t}\}$ ，则复振幅满足

$$[\mathbf{K}_j - \omega^2 \mathbf{M}_j + i\omega (\mathbf{C}_{j,0} + c_j \mathbf{r} \mathbf{r}^\top)] \mathbf{X}_j = \mathbf{F}_j, \quad (36)$$

其中 $\mathbf{C}_{j,0}$ 仅包含浮子的兴波阻尼。两类 PTO 的周期平均功率为

$$\bar{P}_j(c_j) = \frac{1}{2} c_j \omega^2 |\mathbf{r}^\top \mathbf{X}_j|^2, \quad j \in \{z, \theta\}. \quad (37)$$

一阶模型中垂荡与纵摇两个子系统分离，因此联合目标可严格写成

$$\overline{P}(c_z, c_\theta) = \overline{P}_z(c_z) + \overline{P}_\theta(c_\theta). \quad (38)$$

这一分解是由块对角动力学结构导出的精确结果，而非忽略两类 PTO 之间影响的近似。

8.2 解析最优条件

对任一子系统，定义

$$\mathbf{Z}_0 = \mathbf{K} - \omega^2 \mathbf{M} + i\omega \mathbf{C}_0, \quad \alpha = \mathbf{r}^\top \mathbf{Z}_0^{-1} \mathbf{F}, \quad s = \mathbf{r}^\top \mathbf{Z}_0^{-1} \mathbf{r}. \quad (39)$$

应用 Sherman–Morrison 秩一更新公式，得到相对位移复振幅和平均功率

$$\mathbf{r}^\top \mathbf{X}(c) = \frac{\alpha}{1 + i\omega cs}, \quad \overline{P}(c) = \frac{\frac{1}{2}c\omega^2|\alpha|^2}{|1 + i\omega cs|^2}. \quad (40)$$

对 c 求导后，导数符号仅取决于 $1 - \omega^2|s|^2c^2$ ，故无约束正最优点为

$$c_0^* = \frac{1}{\omega|s|}. \quad (41)$$

将两个无约束最优点分别投影到式 (35) 的可行区间，即得联合问题的约束全局最优解。两个子系统在完整可行区间的功率曲线见图 10。

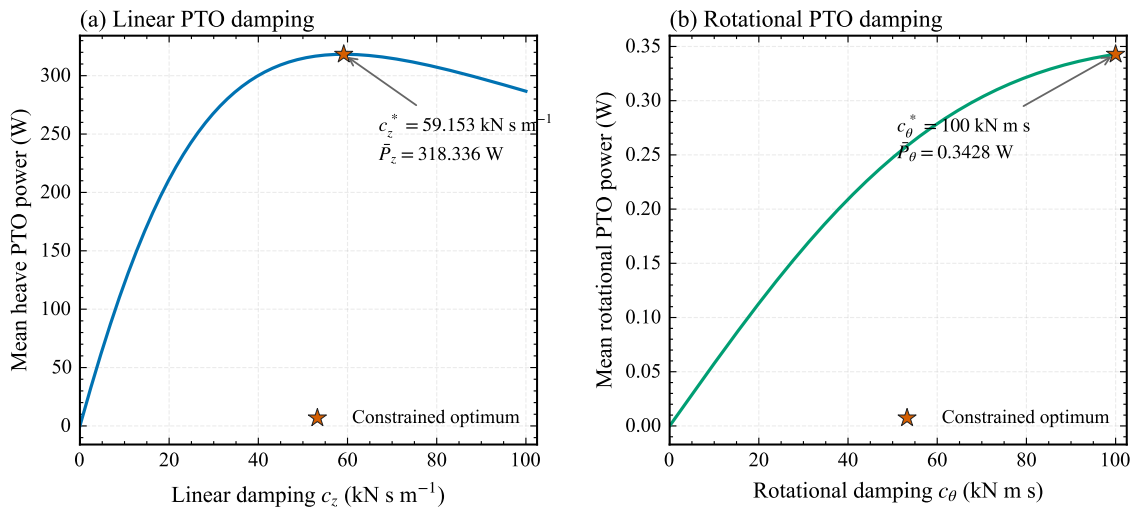


图 10 直线与旋转 PTO 常量阻尼的周期稳态平均功率曲线

直线阻尼的无约束最优点为 59152.916113 N s/m，位于区间内部；旋转阻尼的无约束最优点为 120114.390453 N m s，超过题设上限。因此旋转阻尼在可行区间内随系数增大而增功， $c_\theta = 100000$ 为活跃上边界。

8.3 最优结果与全局性验证

问题四的最优阻尼与平均功率见表 8。

表 8 问题四的最优阻尼与平均功率

指标	最优值
直线阻尼系数 c_z^*	59152.916113 N s/m
旋转阻尼系数 c_θ^*	100000 N m s
直线 PTO 平均功率	318.336440 W
旋转 PTO 平均功率	0.342817 W
最大总平均功率	318.679256 W
单周期总输出能量	1010.966789 J

旋转 PTO 仅占总平均功率的 0.107574%，表明情形 4 中装置输出主要由垂荡相对运动贡献。为排除解析分解和单峰判断带来的路径依赖，本文对两个阻尼区间分别进行 10001 点全域扫描和有界连续优化，并在完整二维参数域内独立执行差分进化搜索。二维搜索经 1554 次目标函数评价得到近似点 (59152.895070, 99999.999943)，总功率为 318.679256467 W，与解析约束最优值一致。图 11 以对数色标给出参数域内相对最优点的功率损失 $\bar{P}_{\max} - \bar{P}$ ，图中色值并非总功率本身。

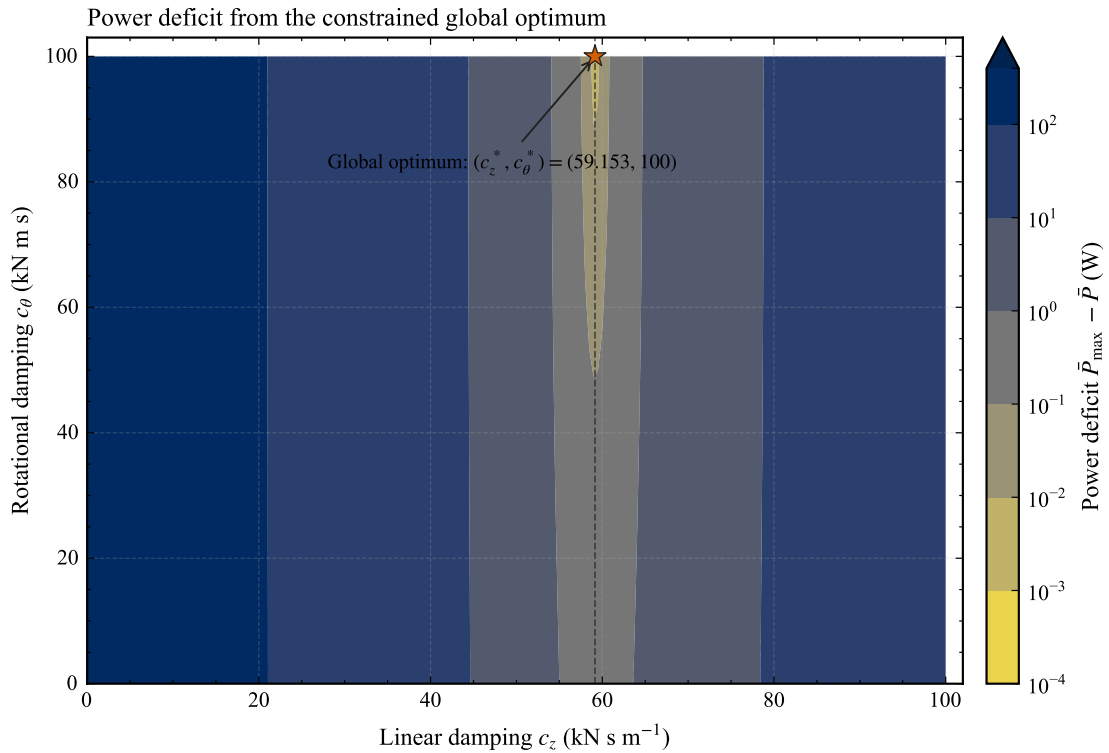


图 11 联合阻尼参数域内相对约束全局最优点的平均功率损失

另对两个最优阻尼系数分别作 1、10、100 和 1000 的可行邻域扰动，所有扰动点的总功率均低于表 8 中的结果，从局部方向进一步验证了内部最优点和活跃边界的正确性。

8.4 最优周期响应与能量闭合

将频域最优解转换为周期初态，再以 DOP853 方法进行时域积分。在完整周期 $T = 3.172364590$ s 内共取 8001 个时刻。最优周期内的相对运动和两类 PTO 瞬时功率如图 12 所示。

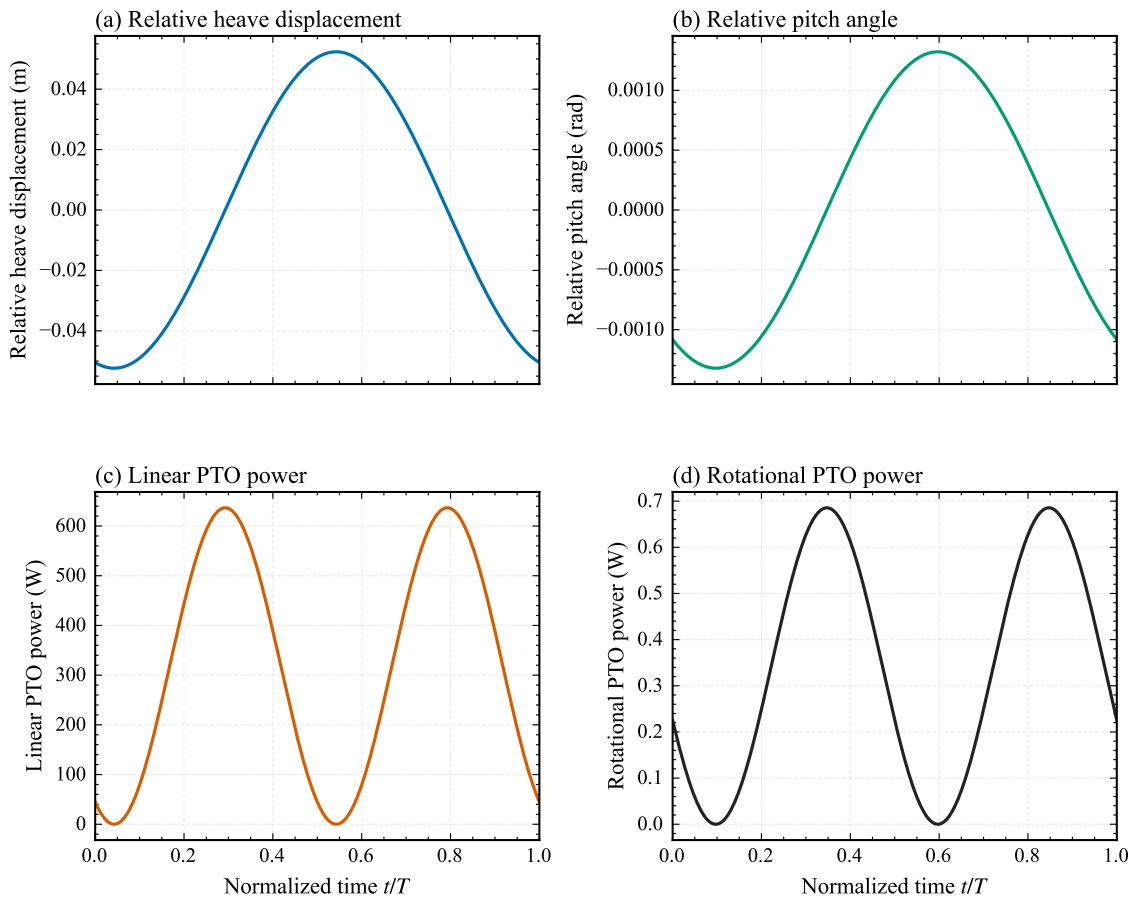


图 12 最优阻尼参数下一个稳定波浪周期的相对运动与 PTO 瞬时功率

垂荡相对位移和速度振幅分别为 0.052380944 m 和 0.103745698 m/s，纵摇相对角和角速度振幅分别为 0.001322054 rad 和 0.002618461 rad/s。直线和旋转 PTO 的瞬时最大功率分别为 636.672832 W 和 0.685634 W。

周期平均能量分配如表 9 所示。

表 9 最优周期的平均功率分配

平均功率项	数值/W
波浪输入	911.991580
垂荡兴波耗散	583.575115
直线 PTO 输出	318.336440
纵摇兴波耗散	9.737209
旋转 PTO 输出	0.342817

周期首尾状态的最大绝对差为 1.78×10^{-15} ，时域总平均功率与频域结果的相对差小于 10^{-13} ，平均能量平衡残差为 5.92×10^{-13} W。弹簧长度保持在 0.149577–0.254338 m，圆柱段吃水保持在 1.249696–2.750325 m，最大绝对纵摇角为 0.055774 rad，均满足本文的物理适用性条件。

九、结论

本文建立了波浪周期激励下浮子–振子系统的二自由度垂荡耦合模型，完成了常量与幂律阻尼下的时域响应计算，并通过收敛、能量和几何范围检验验证了模型可靠性。在此基础上，将平均输出功率统一定义为周期稳态口径，分别得到常量阻尼的唯一内部最优点和幂律阻尼的边界最优点。幂律阻尼的最大平均功率较常量阻尼提高 0.287961%，但比例系数已取到允许上界，说明参数域的边界对最优解具有决定作用。

对垂荡–纵摇联合运动，本文通过壳体几何和平行轴定理补全转动惯量，建立了四自由度线性动力学模型。题给阻尼参数下，装置能量输出主要来自垂荡方向的直线 PTO，旋转 PTO 输出较小。数值积分、矩阵指数解与能量闭合结果一致，并且主响应满足小角度与结构几何范围。壳体敏感性分析同时表明，峰值规模较稳健，但低阻尼纵摇瞬态的相位结论应对转动惯量口径保持谨慎。

在垂荡与纵摇联合优化中，本文利用一阶模型的块对角结构将二维总功率目标严格分解，并以秩一更新得到解析最优条件。最优直线阻尼为 59152.916113 N s/m，最优旋转阻尼取上边界 100000 N m s，最大总平均功率为 318.679256 W。解析推导、穷举扫描、二维差分进化与周期时域能量闭合相互一致，使最终结果同时具备全局最优性、数值可复现性和物理自洽性。

附录 A 支撑材料与结果文件说明

问题一两种阻尼工况的 898 个完整输出时刻已分别写入下列官方格式结果表：

- result1-1.xlsx：常量阻尼完整结果；
- result1-2.xlsx：幂律阻尼完整结果。

正文表 2 和表 3 均直接取自上述文件。问题二的最终参数、功率、周期指标和验收状态汇总于支撑材 q2_final_summary.json。问题三的 733×9 正式结果位于 result3.xlsx，正文表 6 和表 7 的指定时刻数值均取自该工作簿；问题三最终汇总为 q3_final_summary.json。问题四无额外官方结果表，其解析最优解、独立搜索结果、周期响应及验收状态汇总于 q4_final_summary.json。

代码附录按“每问主求解入口”给出，完整工程还包含以下支撑模块：

- code/common/q1_dynamics.py：问题一统一动力学与能量计算；
- code/common/q2_power.py：问题二频域功率和周期射击计算；
- code/common/q3_dynamics.py：问题三和问题四的四自由度动力学；
- code/common/q4_power.py：问题四频域功率与解析最优条件；
- code/q1-code/q4 中的 verify_*.py 和绘图脚本：回归测试、交付核验与正式图生成。

附录 B 问题一核心求解代码

Listing 1: 问题一两类阻尼工况的时域求解与结果导出

```
1  """运行问题一完整 40 周期计算并执行全时域数值验证。"""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from pathlib import Path
8
9  import numpy as np
10
11
12  CODE_DIR = Path(__file__).resolve().parents[1]
13  CONTEST_DIR = CODE_DIR.parent
14  if str(CODE_DIR) not in sys.path:
15      sys.path.insert(0, str(CODE_DIR))
16
17  from common.q1_dynamics import ( # noqa: E402
18      DampingLaw,
```

```

19     Q1Parameters,
20     constant_system_matrices,
21     damping_force,
22     mechanical_energy,
23     output_time_grid,
24     solve_response,
25 )
26
27
28 KEY_TIMES = (10.0, 20.0, 40.0, 60.0, 100.0)
29 STATE_NAMES = ("float_displacement", "float_velocity", "oscillator_displacement",
30               "oscillator_velocity")
31
32 def energy_balance_metrics(solution, params: Q1Parameters) -> dict[str, float]:
33     """计算全时域能量闭合误差和耗散量。"""
34
35     energies = np.array(
36         [mechanical_energy(solution.y[:4, i], params) for i in range(solution.t.size)]
37     )
38     wave_work, radiation_loss, pto_loss = solution.y[4:7]
39     residual = energies - energies[0] - wave_work + radiation_loss + pto_loss
40     scale = np.maximum(
41         1.0,
42         np.abs(energies - energies[0])
43         + np.abs(wave_work)
44         + np.abs(radiation_loss)
45         + np.abs(pto_loss),
46     )
47     return {
48         "max_absolute_residual_J": float(np.max(np.abs(residual))),
49         "max_relative_residual": float(np.max(np.abs(residual) / scale)),
50         "final_mechanical_energy_J": float(energies[-1]),
51         "wave_input_work_J": float(wave_work[-1]),
52         "radiation_loss_J": float(radiation_loss[-1]),
53         "pto_loss_J": float(pto_loss[-1]),
54         "minimum_radiation_loss_J": float(np.min(radiation_loss)),
55         "minimum_pto_loss_J": float(np.min(pto_loss)),
56     }
57
58
59 def convergence_metrics(standard, reference) -> dict[str, object]:
60     """比较正式积分与高精度参考积分的全部输出点。"""
61
62     difference = standard.y[:4] - reference.y[:4]
63     per_state = np.max(np.abs(difference), axis=1)
64     return {

```

```

65     "per_state_max_absolute_difference": {
66         name: float(value) for name, value in zip(STATE_NAMES, per_state)
67     },
68     "overall_max_absolute_difference": float(np.max(per_state)),
69 }
70
71
72 def frequency_domain_validation(params: Q1Parameters) -> dict[str, object]:
73     """以解析稳态作为初值，验证常量阻尼时域方程。"""
74
75     mass, damping, stiffness = constant_system_matrices(params)
76     dynamic_stiffness = (
77         stiffness
78         - params.wave_omega**2 * mass
79         + 1j * params.wave_omega * damping
80     )
81     force = np.array([params.excitation_amplitude, 0.0], dtype=complex)
82     displacement_phasor = np.linalg.solve(dynamic_stiffness, force)
83     initial_state = np.array(
84         [
85             displacement_phasor[0].real,
86             -params.wave_omega * displacement_phasor[0].imag,
87             displacement_phasor[1].real,
88             -params.wave_omega * displacement_phasor[1].imag,
89         ]
90     )
91     times = np.linspace(0.0, 5.0 * params.period, 2001)
92     numerical = solve_response(
93         params,
94         DampingLaw.CONSTANT,
95         (0.0, times[-1]),
96         times,
97         initial_state=initial_state,
98         rtol=1e-11,
99         atol=1e-13,
100        max_step=0.01,
101    )
102    phase = np.exp(1j * params.wave_omega * times)
103    exact = np.vstack(
104        [
105            np.real(displacement_phasor[0] * phase),
106            np.real(1j * params.wave_omega * displacement_phasor[0] * phase),
107            np.real(displacement_phasor[1] * phase),
108            np.real(1j * params.wave_omega * displacement_phasor[1] * phase),
109        ]
110    )
111    error = numerical.y - exact

```

```

112 amplitude_scale = np.maximum(np.max(np.abs(exact), axis=1), 1e-15)
113 relative_errors = np.max(np.abs(error), axis=1) / amplitude_scale
114 return {
115     "displacement_amplitude_m": {
116         "float": float(abs(displacement_phasor[0])),
117         "oscillator": float(abs(displacement_phasor[1])),
118     },
119     "velocity_amplitude_m_per_s": {
120         "float": float(params.wave_omega * abs(displacement_phasor[0])),
121         "oscillator": float(params.wave_omega * abs(displacement_phasor[1])),
122     },
123     "per_state_max_relative_error": {
124         name: float(value) for name, value in zip(STATE_NAMES, relative_errors)
125     },
126     "overall_max_absolute_error": float(np.max(np.abs(error))),
127     "overall_max_relative_error": float(np.max(relative_errors)),
128 }
129
130
131 def response_metrics(solution, params: Q1Parameters, law: DampingLaw) -> dict[str, object]:
132     """汇总响应幅值、相对运动、PTO 功率和关键时刻数值。"""
133
134     states = solution.y[:4]
135     relative_displacement = states[2] - states[0]
136     relative_velocity = states[3] - states[1]
137     cylinder_draft = params.static_cylinder_draft - states[0]
138     spring_length = params.spring_equilibrium_length + relative_displacement
139     pto_power = damping_force(relative_velocity, law, params) * relative_velocity
140     key_values: dict[str, dict[str, float]] = {}
141     for key_time in KEY_TIMES:
142         index = int(round(key_time / 0.2))
143         if not np.isclose(solution.t[index], key_time, atol=1e-12):
144             raise RuntimeError(f"关键时刻 {key_time} s 未正确落在输出网格上")
145         key_values[f"{key_time:.1f}"] = {
146             name: float(states[row, index]) for row, name in enumerate(STATE_NAMES)
147         }
148
149     return {
150         "state_max_absolute": {
151             name: float(value)
152             for name, value in zip(STATE_NAMES, np.max(np.abs(states), axis=1))
153         },
154         "relative_displacement_max_absolute_m": float(
155             np.max(np.abs(relative_displacement))
156         ),
157         "relative_velocity_max_absolute_m_per_s": float(
158             np.max(np.abs(relative_velocity))
159         )
160     }

```

```

159     ),
160     "physical_regime": {
161         "cylinder_draft_min_m": float(np.min(cylinder_draft)),
162         "cylinder_draft_max_m": float(np.max(cylinder_draft)),
163         "cylinder_height_m": params.float_cylinder_height,
164         "spring_length_min_m": float(np.min(spring_length)),
165         "spring_length_max_m": float(np.max(spring_length)),
166     },
167     "pto_power_min_W": float(np.min(pto_power)),
168     "pto_power_max_W": float(np.max(pto_power)),
169     "pto_power_mean_over_full_interval_W": float(np.trapezoid(pto_power, solution.t) /
170 (solution.t[-1] - solution.t[0])),
171     "key_times": key_values,
172 }
173
174 def cycle_to_cycle_metrics(params: Q1Parameters, law: DampingLaw) -> dict[str, object]:
175     """比较第 39、40 个周期的同相位状态，量化瞬态残留。"""
176
177     phase_points = 201
178     times = np.linspace(38.0 * params.period, 40.0 * params.period, 2 * phase_points - 1)
179     solution = solve_response(
180         params,
181         law,
182         (0.0, 40.0 * params.period),
183         times,
184         rtol=1e-10,
185         atol=1e-12,
186         max_step=0.02,
187     )
188     cycle_39 = solution.y[:, :phase_points]
189     cycle_40 = solution.y[:, phase_points - 1 :]
190     absolute_difference = np.max(np.abs(cycle_40 - cycle_39), axis=1)
191     later_cycle_amplitude = np.maximum(np.max(np.abs(cycle_40), axis=1), 1e-15)
192     later_cycle_half_range = 0.5 * (np.max(cycle_40, axis=1) - np.min(cycle_40, axis=1))
193     relative_difference = absolute_difference / later_cycle_amplitude
194     return {
195         "per_state_max_absolute_difference": {
196             name: float(value) for name, value in zip(STATE_NAMES, absolute_difference)
197         },
198         "per_state_relative_to_cycle_40_amplitude": {
199             name: float(value) for name, value in zip(STATE_NAMES, relative_difference)
200         },
201         "cycle_40_max_absolute": {
202             name: float(value) for name, value in zip(STATE_NAMES, later_cycle_amplitude)
203         },
204         "cycle_40_half_peak_to_peak": {

```

```

205         name: float(value) for name, value in zip(STATE_NAMES, later_cycle_half_range)
206     },
207     "overall_max_relative_difference": float(np.max(relative_difference)),
208 }
209
210
211 def run_case(params: Q1Parameters, law: DampingLaw, times: np.ndarray) -> tuple[object,
212     dict[str, object]]:
213     """运行单个阻尼工况并完成能量、收敛和响应检查。"""
214
215     standard = solve_response(
216         params,
217         law,
218         (0.0, 40.0 * params.period),
219         times,
220         track_energy=True,
221         rtol=1e-10,
222         atol=1e-12,
223         max_step=0.02,
224     )
225     reference = solve_response(
226         params,
227         law,
228         (0.0, 40.0 * params.period),
229         times,
230         rtol=1e-12,
231         atol=1e-14,
232         max_step=0.01,
233     )
234     metrics = {
235         "solver": {
236             "standard_nfev": int(standard.nfev),
237             "reference_nfev": int(reference.nfev),
238             "output_points": int(times.size),
239         },
240         "convergence": convergence_metrics(standard, reference),
241         "energy_balance": energy_balance_metrics(standard, params),
242         "response": response_metrics(standard, params, law),
243         "cycle_39_to_40": cycle_to_cycle_metrics(params, law),
244     }
245     return standard, metrics
246
247 def enforce_acceptance(report: dict[str, object]) -> None:
248     """任何验收指标不合格时阻止结果落盘。"""
249
250     for law_name in (DampingLaw.CONSTANT.value, DampingLaw.POWER.value):

```

```

251     case = report["cases"][law_name]
252     convergence = case["convergence"]
253     energy = case["energy_balance"]
254     response = case["response"]
255     physical = response["physical_regime"]
256     if convergence["overall_max_absolute_difference"] >= 1e-7:
257         raise AssertionError(f"{law_name} 全时域收敛未通过")
258     if energy["max_relative_residual"] >= 1e-8:
259         raise AssertionError(f"{law_name} 能量闭合未通过")
260     if energy["minimum_radiation_loss_J"] < -1e-9:
261         raise AssertionError(f"{law_name} 兴波累计耗散出现负值")
262     if energy["minimum_pto_loss_J"] < -1e-9:
263         raise AssertionError(f"{law_name} PTO 累计耗散出现负值")
264     if response["pto_power_min_W"] < -1e-9:
265         raise AssertionError(f"{law_name} PTO 瞬时功率出现负值")
266     if not 0.0 < physical["cylinder_draft_min_m"]:
267         raise AssertionError(f"{law_name} 水线越过圆锥顶端")
268     if not physical["cylinder_draft_max_m"] < physical["cylinder_height_m"]:
269         raise AssertionError(f"{law_name} 水线越过圆柱顶端")
270     if physical["spring_length_min_m"] <= 0.0:
271         raise AssertionError(f"{law_name} 弹簧计算长度出现非正值")
272
273     frequency = report["constant_frequency_domain_validation"]
274     if frequency["overall_max_relative_error"] >= 1e-8:
275         raise AssertionError("常量阻尼频域独立校验未通过")
276
277
278 def main() -> None:
279     params = Q1Parameters()
280     times = output_time_grid(params)
281     case_solutions: dict[str, object] = {}
282     case_reports: dict[str, object] = {}
283
284     for law in DampingLaw:
285         solution, metrics = run_case(params, law, times)
286         case_solutions[law.value] = solution
287         case_reports[law.value] = metrics
288
289     report: dict[str, object] = {
290         "status": "pending_acceptance",
291         "model": "CUMCM 2022 A, question 1",
292         "time_grid": {
293             "start_s": float(times[0]),
294             "end_s": float(times[-1]),
295             "step_s": 0.2,
296             "points": int(times.size),
297             "forty_period_endpoint_s": float(40.0 * params.period),

```

```

298     },
299     "cases": case_reports,
300     "constant_frequency_domain_validation": frequency_domain_validation(params),
301 }
302 enforce_acceptance(report)
303 report["status"] = "passed"
304
305 models_dir = CONTEST_DIR / "results" / "models"
306 models_dir.mkdir(parents=True, exist_ok=True)
307 np.savez_compressed(
308     models_dir / "q1_full_response.npz",
309     time=times,
310     constant=case_solutions[DampingLaw.CONSTANT.value].y[:4],
311     power=case_solutions[DampingLaw.POWER.value].y[:4],
312 )
313 response_payload = {
314     "time": times.tolist(),
315     "constant": case_solutions[DampingLaw.CONSTANT.value].y[:4].tolist(),
316     "power": case_solutions[DampingLaw.POWER.value].y[:4].tolist(),
317 }
318 with (models_dir / "q1_full_response.json").open("w", encoding="utf-8") as stream:
319     json.dump(response_payload, stream, ensure_ascii=False)
320 with (models_dir / "q1_validation.json").open("w", encoding="utf-8") as stream:
321     json.dump(report, stream, ensure_ascii=False, indent=2)
322
323 print(json.dumps(report, ensure_ascii=False, indent=2))
324
325
326 if __name__ == "__main__":
327     main()

```

附录 C 问题二常量阻尼优化代码

Listing 2: 问题二常量阻尼的全区间扫描与频域优化

```

1  """问题二第一部分：常量 PTO 阻尼的一维稳态平均功率优化。"""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from pathlib import Path
8
9  import numpy as np
10 from scipy.optimize import minimize_scalar

```

```

11
12
13 CODE_DIR = Path(__file__).resolve().parents[1]
14 CONTEST_DIR = CODE_DIR.parent
15 if str(CODE_DIR) not in sys.path:
16     sys.path.insert(0, str(CODE_DIR))
17
18 from common.q2_power import ( # noqa: E402
19     constant_mean_power,
20     constant_system_matrices,
21     periodic_mean_power,
22     q2_parameters,
23 )
24
25
26 LOWER_BOUND = 0.0
27 UPPER_BOUND = 100000.0
28 GRID_POINTS = 10001
29
30
31 def analytic_optimum() -> tuple[float, complex]:
32     """由秩一阻尼更新推导常量阻尼的无约束解析最优值。
33
34     令  $Z(c)=Z_0+i\omega c*q*q^T$ , 其中  $q=[1,-1]^T$ 。相对位移
35     复幅值的分母为  $1+i\omega c*s$ ,  $s=q^T Z_0^{-1}*q$ , 故平均
36     功率正比于  $c/|1+i\omega c*s|^2$ , 其内部唯一驻点为
37      $c=1/(\omega*|s|)$ 。
38     """
39
40     params = q2_parameters()
41     mass, damping_zero, stiffness = constant_system_matrices(0.0, params)
42     impedance_zero = (
43         stiffness
44         - params.wave_omega**2 * mass
45         + 1j * params.wave_omega * damping_zero
46     )
47     relative_vector = np.array([1.0, -1.0])
48     receptance = relative_vector @ np.linalg.solve(
49         impedance_zero, relative_vector
50     )
51     optimum = 1.0 / (params.wave_omega * abs(receptance))
52     return float(optimum), complex(receptance)
53
54
55 def count_strict_grid_peaks(power: np.ndarray) -> tuple[int, np.ndarray]:
56     """统计不含边界的严格网格局部极大值。"""
57

```

```

58     peak_indices = np.flatnonzero(
59         (power[1:-1] > power[:-2]) & (power[1:-1] > power[2:])
60     ) + 1
61     return int(peak_indices.size), peak_indices
62
63
64 def enforce_acceptance(report: dict[str, object]) -> None:
65     """对最优位置、单峰性、解析一致性与时域复核执行硬验收。"""
66
67     checks = report["acceptance"]
68     failed = [name for name, passed in checks.items() if not passed]
69     if failed:
70         raise AssertionError("常量阻尼优化验收失败: " + "、".join(failed))
71
72
73 def main() -> None:
74     params = q2_parameters()
75
76     damping_grid = np.linspace(LOWER_BOUND, UPPER_BOUND, GRID_POINTS)
77     power_grid = np.array(
78         [constant_mean_power(value, params).mean_power for value in damping_grid]
79     )
80     grid_best_index = int(np.argmax(power_grid))
81     peak_count, peak_indices = count_strict_grid_peaks(power_grid)
82
83     optimized = minimize_scalar(
84         lambda value: -constant_mean_power(value, params).mean_power,
85         bounds=(LOWER_BOUND, UPPER_BOUND),
86         method="bounded",
87         options={"xatol": 1e-10, "maxiter": 500},
88     )
89     if not optimized.success:
90         raise RuntimeError(f"一维有界优化失败: {optimized.message}")
91
92     optimum_damping = float(optimized.x)
93     frequency_result = constant_mean_power(optimum_damping, params)
94     analytic_damping, receptance = analytic_optimum()
95     analytic_result = constant_mean_power(analytic_damping, params)
96
97     # p=0 时，幂律时域模型严格退化为同一常量阻尼模型。
98     time_result = periodic_mean_power(
99         optimum_damping,
100         0.0,
101         params,
102         convergence_tolerance=1e-10,
103         required_consecutive_cycles=4,
104         rtol=1e-10,

```

```

105     atol=1e-12,
106 )
107
108 perturbation_ratios = np.array([-0.05, -0.01, -0.001, 0.001, 0.01, 0.05])
109 perturbation_damping = optimum_damping * (1.0 + perturbation_ratios)
110 perturbation_power = np.array(
111     [constant_mean_power(value, params).mean_power for value in perturbation_damping]
112 )
113 finite_difference_step = 1.0
114 curvature = (
115     constant_mean_power(optimum_damping + finite_difference_step, params).mean_power
116     - 2.0 * frequency_result.mean_power
117     + constant_mean_power(optimum_damping - finite_difference_step, params).mean_power
118 ) / finite_difference_step**2
119
120 optimizer_analytic_relative_error = abs(
121     optimum_damping - analytic_damping
122 ) / analytic_damping
123 time_frequency_relative_error = abs(
124     time_result.mean_power - frequency_result.mean_power
125 ) / frequency_result.mean_power
126
127 report: dict[str, object] = {
128     "status": "pending_acceptance",
129     "model": "CUMCM 2022 A, question 2, constant PTO damping",
130     "bounds_N_s_per_m": [LOWER_BOUND, UPPER_BOUND],
131     "dense_grid": {
132         "points": GRID_POINTS,
133         "step_N_s_per_m": float(damping_grid[1] - damping_grid[0]),
134         "best_damping_N_s_per_m": float(damping_grid[grid_best_index]),
135         "best_mean_power_W": float(power_grid[grid_best_index]),
136         "strict_interior_peak_count": peak_count,
137         "strict_peak_damping_N_s_per_m": damping_grid[peak_indices].tolist(),
138     },
139     "bounded_optimizer": {
140         "success": bool(optimized.success),
141         "iterations": int(optimized.nit),
142         "function_evaluations": int(optimized.nfev),
143         "damping_N_s_per_m": optimum_damping,
144         "mean_power_W": frequency_result.mean_power,
145         "relative_displacement_amplitude_m": (
146             frequency_result.relative_displacement_amplitude
147         ),
148         "relative_velocity_amplitude_m_per_s": (
149             frequency_result.relative_velocity_amplitude
150         ),
151         "float_displacement_phasor_m": {

```

```

152         "real": float(frequency_result.displacement_phasor[0].real),
153         "imag": float(frequency_result.displacement_phasor[0].imag),
154     },
155     "oscillator_displacement_phasor_m": {
156         "real": float(frequency_result.displacement_phasor[1].real),
157         "imag": float(frequency_result.displacement_phasor[1].imag),
158     },
159     "dynamic_residual_norm_N": frequency_result.dynamic_residual_norm,
160     "local_curvature_W_per_damping_squared": float(curvature),
161 },
162 "analytic_check": {
163     "receptance_real": float(receptance.real),
164     "receptance_imag": float(receptance.imag),
165     "damping_N_s_per_m": analytic_damping,
166     "mean_power_W": analytic_result.mean_power,
167     "optimizer_damping_relative_error": optimizer_analytic_relative_error,
168 },
169 "time_domain_check": {
170     "mean_power_W": time_result.mean_power,
171     "frequency_time_relative_error": time_frequency_relative_error,
172     "cycles_to_convergence": time_result.cycles,
173     "cycle_state_error": time_result.convergence_error,
174     "function_evaluations": time_result.function_evaluations,
175 },
176 "boundary_power_W": {
177     "lower": float(power_grid[0]),
178     "upper": float(power_grid[-1]),
179 },
180 "local_perturbations": [
181     {
182         "relative_damping_change": float(ratio),
183         "damping_N_s_per_m": float(damping),
184         "mean_power_W": float(power),
185         "power_loss_from_optimum_W": float(
186             frequency_result.mean_power - power
187         ),
188     }
189     for ratio, damping, power in zip(
190         perturbation_ratios, perturbation_damping, perturbation_power
191     )
192 ],
193 "acceptance": {
194     "grid_peak_is_interior": 0 < grid_best_index < GRID_POINTS - 1,
195     "grid_has_one_strict_peak": peak_count == 1,
196     "optimizer_is_interior": LOWER_BOUND < optimum_damping < UPPER_BOUND,
197     "all_local_perturbations_are_lower": bool(
198         np.all(perturbation_power < frequency_result.mean_power)

```

```

199         ),
200         "curvature_is_negative": bool(curvature < 0.0),
201         "optimizer_matches_analytic_condition": bool(
202             optimizer_analytic_relative_error < 1e-6
203         ),
204         "frequency_dynamic_residual_passes": bool(
205             frequency_result.dynamic_residual_norm < 1e-8
206         ),
207         "time_domain_matches_frequency_domain": bool(
208             time_frequency_relative_error < 1e-8
209         ),
210     },
211 }
212 enforce_acceptance(report)
213 report["status"] = "passed"
214
215 models_dir = CONTEST_DIR / "results" / "models"
216 models_dir.mkdir(parents=True, exist_ok=True)
217 np.savez_compressed(
218     models_dir / "q2_constant_power_scan.npz",
219     damping=damping_grid,
220     mean_power=power_grid,
221 )
222 with (models_dir / "q2_constant_optimization.json").open(
223     "w", encoding="utf-8"
224 ) as stream:
225     json.dump(report, stream, ensure_ascii=False, indent=2)
226
227 print(json.dumps(report, ensure_ascii=False, indent=2))
228
229
230 if __name__ == "__main__":
231     main()

```

附录 D 问题二幂律阻尼优化代码

Listing 3: 问题二幂律阻尼的边界精修与周期独立验证

```

1  """问题二幂律阻尼最终候选的严格、独立数值复核。"""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from pathlib import Path

```

```

8  from time import perf_counter
9
10 import numpy as np
11 from scipy.optimize import minimize_scalar
12
13
14 CODE_DIR = Path(__file__).resolve().parents[1]
15 CONTEST_DIR = CODE_DIR.parent
16 if str(CODE_DIR) not in sys.path:
17     sys.path.insert(0, str(CODE_DIR))
18
19 from common.q2_power import ( # noqa: E402
20     periodic_mean_power,
21     pto_power,
22     q2_parameters,
23     sample_period_from_state,
24     shooting_mean_power,
25 )
26
27
28 COEFFICIENT = 100000.0
29 STRICT_SETTINGS = {
30     "root_tolerance": 1e-12,
31     "periodicity_tolerance": 5e-10,
32     "maximum_root_evaluations": 200,
33     "rtol": 2e-12,
34     "atol": 2e-14,
35     "max_step_fraction": 1.0 / 240.0,
36 }
37
38
39 def relative_error(value: float, reference: float) -> float:
40     return abs(value - reference) / max(abs(reference), 1e-15)
41
42
43 def enforce_acceptance(report: dict[str, object]) -> None:
44     failed = [name for name, passed in report["acceptance"].items() if not passed]
45     if failed:
46         raise AssertionError("幂律最终复核失败: " + "、".join(failed))
47
48
49 def main() -> None:
50     params = q2_parameters()
51     boundary_report_path = (
52         CONTEST_DIR
53         / "results"
54         / "models"

```

```

55     / "q2_nonlinear_boundary_optimization.json"
56 )
57 with boundary_report_path.open("r", encoding="utf-8") as stream:
58     boundary_report = json.load(stream)
59     prior_candidate = boundary_report["bounded_refinement"]["candidate"]
60     prior_exponent = float(prior_candidate["exponent"])
61     prior_state = np.asarray(prior_candidate["periodic_state"], dtype=float)
62
63     cache: dict[tuple[float, float], object] = {}
64
65     def strict_result(
66         coefficient: float,
67         exponent: float,
68         initial_state: np.ndarray | None = prior_state,
69     ):
70         key = (round(float(coefficient), 8), round(float(exponent), 13))
71         if key not in cache:
72             cache[key] = shooting_mean_power(
73                 coefficient,
74                 exponent,
75                 params,
76                 initial_state=initial_state,
77                 **STRICT_SETTINGS,
78             )
79         return cache[key]
80
81     refinement_start = perf_counter()
82     optimized = minimize_scalar(
83         lambda exponent: -strict_result(COEFFICIENT, exponent).mean_power,
84         bounds=(prior_exponent - 0.005, prior_exponent + 0.005),
85         method="bounded",
86         options={"xatol": 2e-10, "maxiter": 100},
87     )
88     if not optimized.success:
89         raise RuntimeError(f"最终指数精修失败: {optimized.message}")
90     final_exponent = float(optimized.x)
91     final_shooting = strict_result(COEFFICIENT, final_exponent)
92     refinement_seconds = perf_counter() - refinement_start
93
94     exponent_changes = np.array(
95         [-5e-4, -1e-4, -5e-5, -1e-5, 1e-5, 5e-5, 1e-4, 5e-4]
96     )
97     exponent_results = [
98         strict_result(COEFFICIENT, final_exponent + change)
99         for change in exponent_changes
100     ]
101     exponent_power = np.array([result.mean_power for result in exponent_results])

```

```

102
103 derivative_step = 1e-5
104 power_minus = strict_result(
105     COEFFICIENT, final_exponent - derivative_step
106 ).mean_power
107 power_plus = strict_result(
108     COEFFICIENT, final_exponent + derivative_step
109 ).mean_power
110 central_derivative = (power_plus - power_minus) / (2.0 * derivative_step)
111 central_curvature = (
112     power_plus - 2.0 * final_shooting.mean_power + power_minus
113 ) / derivative_step**2
114
115 coefficient_inward = np.array([99990.0, 99900.0, 99000.0])
116 coefficient_results = [
117     strict_result(coefficient, final_exponent)
118     for coefficient in coefficient_inward
119 ]
120 coefficient_power = np.array(
121     [result.mean_power for result in coefficient_results]
122 )
123 inward_derivative = (
124     final_shooting.mean_power - coefficient_power[0]
125 ) / (COEFFICIENT - coefficient_inward[0])
126
127 independent_shooting = shooting_mean_power(
128     COEFFICIENT,
129     final_exponent,
130     params,
131     initial_state=np.zeros(4),
132     **STRICT_SETTINGS,
133 )
134
135 cold_start_time = perf_counter()
136 cold_start = periodic_mean_power(
137     COEFFICIENT,
138     final_exponent,
139     params,
140     initial_state=np.zeros(4),
141     convergence_tolerance=1e-8,
142     required_consecutive_cycles=5,
143     minimum_cycles=10,
144     maximum_cycles=3000,
145     rtol=1e-9,
146     atol=1e-11,
147     max_step_fraction=1.0 / 60.0,
148 )

```

```

149 cold_start_seconds = perf_counter() - cold_start_time
150
151 times, states, instantaneous_power = sample_period_from_state(
152     final_shooting.periodic_state,
153     COEFFICIENT,
154     final_exponent,
155     params,
156     samples=8001,
157     rtol=1e-12,
158     atol=1e-14,
159 )
160 quadrature_energy = float(np.trapezoid(instantaneous_power, times))
161 quadrature_mean_power = quadrature_energy / params.period
162 relative_velocity = states[3] - states[1]
163 relative_displacement = states[2] - states[0]
164 cylinder_draft = params.static_cylinder_draft - states[0]
165 spring_length = params.spring_equilibrium_length + relative_displacement
166 independently_sampled_power = np.asarray(
167     pto_power(relative_velocity, COEFFICIENT, final_exponent), dtype=float
168 )
169
170 shooting_cold_power_error = relative_error(
171     final_shooting.mean_power, cold_start.mean_power
172 )
173 shooting_quadrature_power_error = relative_error(
174     final_shooting.mean_power, quadrature_mean_power
175 )
176 shooting_state_error = float(
177     np.linalg.norm(
178         final_shooting.periodic_state - independent_shooting.periodic_state
179     )
180     / (1.0 + np.linalg.norm(final_shooting.periodic_state))
181 )
182 cold_state_error = float(
183     np.linalg.norm(final_shooting.periodic_state - cold_start.periodic_state)
184     / (1.0 + np.linalg.norm(final_shooting.periodic_state))
185 )
186
187 ridge_report_path = (
188     CONTEST_DIR / "results" / "models" / "q2_nonlinear_ridge_refinement.json"
189 )
190 with ridge_report_path.open("r", encoding="utf-8") as stream:
191     ridge_report = json.load(stream)
192 internal_audit_best = max(
193     run["candidate"]["mean_power_W"]
194     for run in ridge_report["interior_multistart_audit"]["runs"]
195 )

```

```

196
197 report: dict[str, object] = {
198     "status": "pending_acceptance",
199     "model": "CUMCM 2022 A, question 2, final power-law PTO result",
200     "final_result": {
201         "coefficient": COEFFICIENT,
202         "exponent": final_exponent,
203         "mean_power_W": final_shooting.mean_power,
204         "pto_energy_per_period_J": final_shooting.pto_energy_one_cycle,
205         "period_s": params.period,
206         "periodic_state": final_shooting.periodic_state.tolist(),
207         "periodicity_error": final_shooting.periodicity_error,
208         "root_success": final_shooting.root_success,
209     },
210     "strict_refinement": {
211         "prior_candidate_exponent": prior_exponent,
212         "exponent_change": final_exponent - prior_exponent,
213         "optimizer_success": bool(optimized.success),
214         "iterations": int(optimized.nit),
215         "function_evaluations": int(optimized.nfev),
216         "unique_strict_evaluations_including_checks": len(cache),
217         "wall_time_s": refinement_seconds,
218         "central_derivative_W_per_exponent": float(central_derivative),
219         "central_curvature_W_per_exponent_squared": float(central_curvature),
220     },
221     "exponent_perturbations": [
222         {
223             "change": float(change),
224             "exponent": float(final_exponent + change),
225             "mean_power_W": float(result.mean_power),
226             "power_loss_W": float(
227                 final_shooting.mean_power - result.mean_power
228             ),
229             "periodicity_error": float(result.periodicity_error),
230         }
231         for change, result in zip(exponent_changes, exponent_results)
232     ],
233     "coefficient_inward_checks": [
234         {
235             "coefficient": float(coefficient),
236             "exponent_fixed_at_final": final_exponent,
237             "mean_power_W": float(result.mean_power),
238             "power_loss_W": float(
239                 final_shooting.mean_power - result.mean_power
240             ),
241             "periodicity_error": float(result.periodicity_error),
242         }

```

```

243         for coefficient, result in zip(
244             coefficient_inward, coefficient_results
245         )
246     ],
247     "coefficient_one_sided_derivative_W_per_coefficient": float(
248         inward_derivative
249     ),
250     "independent_zero_guess_shooting": {
251         "mean_power_W": independent_shooting.mean_power,
252         "periodicity_error": independent_shooting.periodicity_error,
253         "state_relative_difference": shooting_state_error,
254         "power_relative_difference": relative_error(
255             final_shooting.mean_power, independent_shooting.mean_power
256         ),
257     },
258     "cold_start_long_transient": {
259         "mean_power_W": cold_start.mean_power,
260         "cycles_to_convergence": cold_start.cycles,
261         "cycle_state_error": cold_start.convergence_error,
262         "function_evaluations": cold_start.function_evaluations,
263         "wall_time_s": cold_start_seconds,
264         "state_relative_difference_from_shooting": cold_state_error,
265         "power_relative_difference_from_shooting": shooting_cold_power_error,
266     },
267     "dense_period_quadrature": {
268         "samples": int(times.size),
269         "mean_power_W": quadrature_mean_power,
270         "energy_J": quadrature_energy,
271         "power_relative_difference_from_augmented_state": (
272             shooting_quadrature_power_error
273         ),
274         "minimum_instantaneous_power_W": float(
275             np.min(instantaneous_power)
276         ),
277         "maximum_instantaneous_power_W": float(
278             np.max(instantaneous_power)
279         ),
280         "independent_power_array_max_difference_W": float(
281             np.max(np.abs(instantaneous_power - independently_sampled_power))
282         ),
283     },
284     "steady_response": {
285         "float_displacement_min_m": float(np.min(states[0])),
286         "float_displacement_max_m": float(np.max(states[0])),
287         "oscillator_displacement_min_m": float(np.min(states[2])),
288         "oscillator_displacement_max_m": float(np.max(states[2])),
289         "relative_displacement_max_absolute_m": float(

```

```

290         np.max(np.abs(relative_displacement))
291     ),
292     "relative_velocity_max_absolute_m_per_s": float(
293         np.max(np.abs(relative_velocity))
294     ),
295     "cylinder_draft_min_m": float(np.min(cylinder_draft)),
296     "cylinder_draft_max_m": float(np.max(cylinder_draft)),
297     "spring_length_min_m": float(np.min(spring_length)),
298     "spring_length_max_m": float(np.max(spring_length)),
299 },
300 "global_comparison": {
301     "best_internal_audit_power_W": float(internal_audit_best),
302     "final_boundary_advantage_W": float(
303         final_shooting.mean_power - internal_audit_best
304     ),
305 },
306 "acceptance": {
307     "strict_optimizer_succeeded": bool(optimized.success),
308     "final_exponent_is_interior": 0.0 < final_exponent < 1.0,
309     "final_coefficient_is_active_upper_bound": COEFFICIENT == 100000.0,
310     "final_periodicity_error_passes": bool(
311         final_shooting.periodicity_error
312         <= STRICT_SETTINGS["periodicity_tolerance"]
313     ),
314     "all_exponent_perturbations_are_lower": bool(
315         np.all(exponent_power < final_shooting.mean_power)
316     ),
317     "exponent_curvature_is_negative": bool(central_curvature < 0.0),
318     "exponent_derivative_is_small": bool(abs(central_derivative) < 5e-3),
319     "all_inward_coefficient_checks_are_lower": bool(
320         np.all(coefficient_power < final_shooting.mean_power)
321     ),
322     "coefficient_one_sided_derivative_is_positive": bool(
323         inward_derivative > 0.0
324     ),
325     "independent_shooting_state_matches": bool(
326         shooting_state_error < 1e-8
327     ),
328     "independent_shooting_power_matches": bool(
329         relative_error(
330             final_shooting.mean_power, independent_shooting.mean_power
331         )
332         < 1e-9
333     ),
334     "cold_start_converged": bool(
335         cold_start.convergence_error <= 1e-8
336     ),

```

```

337     "cold_start_state_matches": bool(cold_state_error < 1e-6),
338     "cold_start_power_matches": bool(shooting_cold_power_error < 1e-7),
339     "dense_quadrature_matches_augmented_energy": bool(
340         shooting_quadrature_power_error < 1e-8
341     ),
342     "instantaneous_power_is_nonnegative": bool(
343         np.min(instantaneous_power) >= -1e-10
344     ),
345     "physical_waterline_regime_passes": bool(
346         np.min(cylinder_draft) > 0.0
347         and np.max(cylinder_draft) < params.float_cylinder_height
348     ),
349     "spring_length_is_positive": bool(np.min(spring_length) > 0.0),
350     "final_boundary_power_exceeds_internal_audit": bool(
351         final_shooting.mean_power > internal_audit_best
352     ),
353 },
354 }
355 enforce_acceptance(report)
356 report["status"] = "passed"
357 report["final_answer_ready"] = True
358
359 models_dir = CONTEST_DIR / "results" / "models"
360 np.savez_compressed(
361     models_dir / "q2_nonlinear_final_period.npz",
362     time=times,
363     state=states,
364     instantaneous_power=instantaneous_power,
365     relative_displacement=relative_displacement,
366     relative_velocity=relative_velocity,
367 )
368 with (models_dir / "q2_nonlinear_final_validation.json").open(
369     "w", encoding="utf-8"
370 ) as stream:
371     json.dump(report, stream, ensure_ascii=False, indent=2)
372
373 print(json.dumps(report, ensure_ascii=False, indent=2))
374
375
376 if __name__ == "__main__":
377     main()

```

附录 E 问题三核心求解与验证代码

Listing 4: 问题三四自由度响应、能量闭合与壳体敏感性验证

```

1  """问题三完整 40 周期数值验证；不生成正式结果工作簿。"""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from dataclasses import replace
8  from pathlib import Path
9  from time import perf_counter
10
11 import numpy as np
12
13
14 CODE_DIR = Path(__file__).resolve().parents[1]
15 if str(CODE_DIR) not in sys.path:
16     sys.path.insert(0, str(CODE_DIR))
17
18 from common.q3_dynamics import ( # noqa: E402
19     FloatShellConvention,
20     Q3Parameters,
21     diagnostic_time_grid,
22     matrix_exponential_response,
23     output_time_grid,
24     solve_response,
25     system_matrices,
26 )
27
28
29 STATE_NAMES = (
30     "float_heave_displacement",
31     "float_heave_velocity",
32     "oscillator_heave_displacement",
33     "oscillator_heave_velocity",
34     "float_pitch_displacement",
35     "float_pitch_velocity",
36     "oscillator_pitch_displacement",
37     "oscillator_pitch_velocity",
38 )
39
40 SAFE_SCALES = np.array([1e-3] * 4 + [1e-4] * 4, dtype=float)
41
42 def channel_error(candidate: np.ndarray, reference: np.ndarray) -> dict[str, object]:
43     """返回逐状态最大误差及统一归一化误差。"""
44
45     maximum = np.max(np.abs(candidate - reference), axis=1)

```

```

46 rms = np.sqrt(np.mean((candidate - reference) ** 2, axis=1))
47 scales = np.maximum(np.max(np.abs(reference), axis=1), SAFE_SCALES)
48 normalized = maximum / scales
49 return {
50     "channel_max_absolute": dict(zip(STATE_NAMES, maximum.tolist(), strict=True)),
51     "channel_rms": dict(zip(STATE_NAMES, rms.tolist(), strict=True)),
52     "channel_max_normalized": dict(
53         zip(STATE_NAMES, normalized.tolist(), strict=True)
54     ),
55     "global_max_absolute": float(np.max(maximum)),
56     "global_max_normalized": float(np.max(normalized)),
57 }
58
59
60 def state_energy(states: np.ndarray, params: Q3Parameters) -> np.ndarray:
61     """向量化计算八状态序列的机械能。"""
62
63     mass, _, stiffness = system_matrices(params)
64     positions = states[[0, 2, 4, 6]]
65     velocities = states[[1, 3, 5, 7]]
66     kinetic = 0.5 * np.einsum("in,ij,jn->n", velocities, mass, velocities)
67     potential = 0.5 * np.einsum("in,ij,jn->n", positions, stiffness, positions)
68     return kinetic + potential
69
70
71 def peak_dictionary(states: np.ndarray) -> dict[str, float]:
72     return dict(
73         zip(STATE_NAMES, np.max(np.abs(states), axis=1).tolist(), strict=True)
74     )
75
76
77 def main() -> None:
78     params = Q3Parameters()
79     output_times = output_time_grid(params)
80     diagnostic_times = diagnostic_time_grid(params, step=0.01)
81     strict_atol = np.array([2e-14] * 4 + [2e-15] * 4, dtype=float)
82     timings: dict[str, float] = {}
83
84     print("[1/6] main 40-period integration", flush=True)
85     start = perf_counter()
86     main_solution = solve_response(
87         params,
88         (0.0, params.forty_period_end),
89         output_times,
90         rtol=1e-10,
91         max_step=0.01,
92     )

```

```

93     timings["main_integration_seconds"] = perf_counter() - start
94
95     print("[2/6] strict reference integration", flush=True)
96     start = perf_counter()
97     reference_solution = solve_response(
98         params,
99         (0.0, params.forty_period_end),
100         output_times,
101         rtol=2e-12,
102         atol=strict_atol,
103         max_step=0.0025,
104     )
105     timings["reference_integration_seconds"] = perf_counter() - start
106
107     print("[3/6] matrix-exponential reference on official grid", flush=True)
108     start = perf_counter()
109     exact_solution = matrix_exponential_response(output_times, params)
110     exact_endpoint = matrix_exponential_response([params.forty_period_end], params)[: , 0]
111     timings["matrix_exponential_seconds"] = perf_counter() - start
112
113     print("[4/6] augmented-energy integration on dense grid", flush=True)
114     start = perf_counter()
115     energy_solution = solve_response(
116         params,
117         (0.0, params.forty_period_end),
118         diagnostic_times,
119         rtol=1e-10,
120         max_step=0.01,
121         track_energy=True,
122     )
123     timings["energy_dense_integration_seconds"] = perf_counter() - start
124
125     print("[5/6] physical-range and energy checks", flush=True)
126     dense_states = energy_solution.y[:8]
127     energies = state_energy(dense_states, params)
128     input_work = energy_solution.y[8]
129     losses = energy_solution.y[9:13]
130     energy_residual = energies - input_work + np.sum(losses, axis=0)
131     energy_scale = max(
132         1.0,
133         float(np.max(np.abs(energies))),
134         float(np.max(np.abs(input_work))),
135         float(np.max(np.sum(losses, axis=0))),
136     )
137     energy_relative_residual = float(
138         np.max(np.abs(energy_residual)) / energy_scale
139     )

```

```

140     spring_lengths = (
141         params.spring_equilibrium_length + dense_states[2] - dense_states[0]
142     )
143     maximum_pitch = float(np.max(np.abs(dense_states[[4, 6]])))
144     output_indices = np rint(output_times / 0.01).astype(int)
145     dense_on_output = dense_states[:, output_indices]
146
147     print("[6/6] alternative-shell sensitivity", flush=True)
148     alternative = replace(
149         params, shell_convention=FloatShellConvention.LATERAL_ONLY
150     )
151     start = perf_counter()
152     alternative_solution = solve_response(
153         alternative,
154         (0.0, alternative.forty_period_end),
155         diagnostic_times,
156         rtol=1e-10,
157         max_step=0.01,
158     )
159     timings["alternative_dense_integration_seconds"] = perf_counter() - start
160     alternative_states = alternative_solution.y
161
162     main_reference_error = channel_error(main_solution.y, reference_solution.y)
163     main_exact_error = channel_error(main_solution.y, exact_solution)
164     reference_exact_error = channel_error(reference_solution.y, exact_solution)
165     augmented_main_error = channel_error(dense_on_output, main_solution.y)
166
167     sensitivity_maximum = np.max(
168         np.abs(alternative_states - dense_states), axis=1
169     )
170     main_peaks = np.max(np.abs(dense_states), axis=1)
171     alternative_peaks = np.max(np.abs(alternative_states), axis=1)
172     peak_relative_change = np.divide(
173         alternative_peaks - main_peaks,
174         np.maximum(main_peaks, SAFE_SCALES),
175     )
176
177     key_indices = {
178         str(time): int(np.flatnonzero(np.isclose(output_times, time))[0])
179         for time in (10.0, 20.0, 40.0, 60.0, 100.0)
180     }
181     key_values = {
182         time: dict(
183             zip(
184                 STATE_NAMES,
185                 main_solution.y[:, index].tolist(),
186                 strict=True,

```

```

187         )
188     )
189     for time, index in key_indices.items()
190 }
191
192 checks = {
193     "official_grid": bool(
194         output_times.size == 733
195         and output_times[0] == 0.0
196         and np.isclose(output_times[-1], 146.4)
197     ),
198     "all_main_values_finite": bool(np.all(np.isfinite(main_solution.y))),
199     "all_dense_values_finite": bool(np.all(np.isfinite(dense_states))),
200     "all_alternative_values_finite": bool(
201         np.all(np.isfinite(alternative_states))
202     ),
203     "main_reference_convergence": bool(
204         main_reference_error["global_max_normalized"] <= 5e-8
205     ),
206     "main_matrix_exponential": bool(
207         main_exact_error["global_max_normalized"] <= 5e-8
208     ),
209     "reference_matrix_exponential": bool(
210         reference_exact_error["global_max_normalized"] <= 5e-9
211     ),
212     "augmented_matches_main": bool(
213         augmented_main_error["global_max_normalized"] <= 5e-8
214     ),
215     "energy_balance": bool(energy_relative_residual <= 1e-8),
216     "losses_nonnegative": bool(float(np.min(losses)) >= -1e-10),
217     "losses_monotone": bool(
218         float(np.min(np.diff(losses, axis=1))) >= -1e-9
219     ),
220     "spring_length_positive": bool(float(np.min(spring_lengths)) > 0.0),
221     "small_angle_warning_not_triggered": bool(maximum_pitch <= 0.2),
222 }
223 checks["all_passed"] = all(checks.values())
224
225 result = {
226     "status": "passed" if checks["all_passed"] else "failed",
227     "shell_convention": params.shell_convention.value,
228     "wave_period": params.period,
229     "forty_period_end": params.forty_period_end,
230     "official_grid": {
231         "count": int(output_times.size),
232         "first": float(output_times[0]),
233         "last": float(output_times[-1]),

```

```

234         "step": 0.2,
235     },
236     "diagnostic_grid": {
237         "count": int(diagnostic_times.size),
238         "first": float(diagnostic_times[0]),
239         "last": float(diagnostic_times[-1]),
240         "regular_step": 0.01,
241     },
242     "timings": timings,
243     "errors": {
244         "main_vs_reference": main_reference_error,
245         "main_vs_matrix_exponential": main_exact_error,
246         "reference_vs_matrix_exponential": reference_exact_error,
247         "augmented_vs_main": augmented_main_error,
248         "energy_relative_residual": energy_relative_residual,
249         "energy_absolute_residual": float(np.max(np.abs(energy_residual))),
250     },
251     "physical_ranges": {
252         "main_peak_absolute": peak_dictionary(dense_states),
253         "minimum_spring_length": float(np.min(spring_lengths)),
254         "maximum_spring_length": float(np.max(spring_lengths)),
255         "maximum_absolute_pitch": maximum_pitch,
256         "exact_state_at_40_period_end": dict(
257             zip(STATE_NAMES, exact_endpoint.tolist(), strict=True)
258         ),
259     },
260     "energy_at_40_period_end": {
261         "mechanical": float(energies[-1]),
262         "wave_input_work": float(input_work[-1]),
263         "heave_radiation_loss": float(losses[0, -1]),
264         "linear_pto_output": float(losses[1, -1]),
265         "pitch_radiation_loss": float(losses[2, -1]),
266         "rotational_pto_output": float(losses[3, -1]),
267         "balance_residual": float(energy_residual[-1]),
268     },
269     "sensitivity": {
270         "alternative_shell_convention": alternative.shell_convention.value,
271         "channel_max_absolute_difference": dict(
272             zip(STATE_NAMES, sensitivity_maximum.tolist(), strict=True)
273         ),
274         "main_peak_absolute": dict(
275             zip(STATE_NAMES, main_peaks.tolist(), strict=True)
276         ),
277         "alternative_peak_absolute": dict(
278             zip(STATE_NAMES, alternative_peaks.tolist(), strict=True)
279         ),
280         "peak_relative_change": dict(

```

```

281         zip(STATE_NAMES, peak_relative_change.tolist(), strict=True)
282     ),
283     },
284     "key_time_values": key_values,
285     "checks": checks,
286 }
287 print(json.dumps(result, ensure_ascii=True, indent=2), flush=True)
288 if not checks["all_passed"]:
289     raise SystemExit(1)
290
291
292 if __name__ == "__main__":
293     main()

```

附录 F 问题四联合阻尼优化代码

Listing 5: 问题四解析优化、二维独立搜索与周期时域复核

```

1  """问题四：直线与旋转常量阻尼的全域优化和最终时域复核。"""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from pathlib import Path
8
9  import numpy as np
10 from scipy.integrate import simpson
11 from scipy.optimize import differential_evolution, minimize_scalar
12
13
14 CODE_DIR = Path(__file__).resolve().parents[1]
15 CONTEST_DIR = CODE_DIR.parent
16 if str(CODE_DIR) not in sys.path:
17     sys.path.insert(0, str(CODE_DIR))
18
19 from common.q3_dynamics import instantaneous_powers, mechanical_energy, solve_response #
    noqa: E402
20 from common.q4_power import ( # noqa: E402
21     AxisPowerResult,
22     axis_mean_power,
23     constrained_analytic_optimum,
24     q4_parameters,
25     total_mean_power,
26     unconstrained_optimal_damping,

```

```

27 )
28
29
30 def result_payload(result: AxisPowerResult) -> dict[str, object]:
31     return {
32         "damping": result.damping,
33         "mean_power_W": result.mean_power,
34         "relative_displacement_amplitude": result.relative_displacement_amplitude,
35         "relative_velocity_amplitude": result.relative_velocity_amplitude,
36         "dynamic_residual": result.dynamic_residual,
37         "displacement_phasor_real": result.displacement_phasor.real.tolist(),
38         "displacement_phasor_imag": result.displacement_phasor.imag.tolist(),
39     }
40
41
42 def bounded_axis_optimum(axis: str, params) -> AxisPowerResult:
43     optimized = minimize_scalar(
44         lambda value: -axis_mean_power(axis, float(value), params).mean_power,
45         bounds=(0.0, 100000.0),
46         method="bounded",
47         options={"xatol": 1e-9, "maxiter": 1000},
48     )
49     if not optimized.success:
50         raise RuntimeError(f"{axis} 有界优化失败: {optimized.message}")
51     candidates = [0.0, float(optimized.x), 100000.0]
52     return max(
53         (axis_mean_power(axis, value, params) for value in candidates),
54         key=lambda result: result.mean_power,
55     )
56
57
58 def periodic_initial_state(heave: AxisPowerResult, pitch: AxisPowerResult, omega: float) ->
    np.ndarray:
59     initial = np.zeros(8)
60     for offset, response in ((0, heave.displacement_phasor), (4, pitch.displacement_phasor)):
61         initial[offset] = response[0].real
62         initial[offset + 1] = (1j * omega * response[0]).real
63         initial[offset + 2] = response[1].real
64         initial[offset + 3] = (1j * omega * response[1]).real
65     return initial
66
67
68 def main() -> None:
69     models_dir = CONTEST_DIR / "results" / "models"
70     models_dir.mkdir(parents=True, exist_ok=True)
71     params = q4_parameters()
72

```

```

73 scan_damping = np.linspace(0.0, 100000.0, 10001)
74 heave_scan = np.array(
75     [axis_mean_power("heave", value, params).mean_power for value in scan_damping]
76 )
77 pitch_scan = np.array(
78     [axis_mean_power("pitch", value, params).mean_power for value in scan_damping]
79 )
80 np.savez_compressed(
81     models_dir / "q4_power_curves.npz",
82     damping=scan_damping,
83     heave_power=heave_scan,
84     pitch_power=pitch_scan,
85 )
86
87 analytic_heave = constrained_analytic_optimum("heave", params)
88 analytic_pitch = constrained_analytic_optimum("pitch", params)
89 bounded_heave = bounded_axis_optimum("heave", params)
90 bounded_pitch = bounded_axis_optimum("pitch", params)
91 final_linear = analytic_heave.damping
92 final_rotational = analytic_pitch.damping
93 final_total = total_mean_power(final_linear, final_rotational, params)
94
95 joint = differential_evolution(
96     lambda values: -total_mean_power(float(values[0]), float(values[1]), params),
97     bounds=((0.0, 100000.0), (0.0, 100000.0)),
98     seed=202208,
99     tol=1e-11,
100     atol=1e-11,
101     polish=True,
102     workers=1,
103     updating="immediate",
104 )
105 if not joint.success:
106     raise RuntimeError(f"二维独立优化失败: {joint.message}")
107
108 final_params = q4_parameters(
109     linear_damping=final_linear,
110     rotational_damping=final_rotational,
111 )
112 time = np.linspace(0.0, params.period, 8001)
113 initial = periodic_initial_state(analytic_heave, analytic_pitch, params.wave_omega)
114 solution = solve_response(
115     final_params,
116     (0.0, params.period),
117     time,
118     initial_state=initial,
119     rtol=1e-11,

```

```

120     max_step=params.period / 3000.0,
121 )
122 powers = np.column_stack(
123     [instantaneous_powers(t, solution.y[:, i], final_params) for i, t in enumerate(time)]
124 )
125 mean_power_components = simpson(powers, x=time, axis=1) / params.period
126 time_total_pto = float(mean_power_components[2] + mean_power_components[4])
127 periodic_closure = float(np.max(np.abs(solution.y[:, -1] - solution.y[:, 0])))
128 mechanical_change = mechanical_energy(solution.y[:, -1], final_params) - mechanical_energy(
129     solution.y[:, 0], final_params
130 )
131 energy_average_residual = float(
132     mean_power_components[0]
133     - mean_power_components[1]
134     - mean_power_components[2]
135     - mean_power_components[3]
136     - mean_power_components[4]
137 )
138 relative_heave = solution.y[2] - solution.y[0]
139 spring_length = params.spring_equilibrium_length + relative_heave
140 maximum_pitch = float(np.max(np.abs(solution.y[[4, 6]])))
141 static_cylinder_draft = (
142     (params.float_mass + params.oscillator_mass) / params.seawater_density
143     - np.pi * params.float_radius**2 * params.float_cone_height / 3.0
144 ) / (np.pi * params.float_radius**2)
145 draft = static_cylinder_draft - solution.y[0]
146
147 neighbor_checks: dict[str, float] = {}
148 for delta in (1.0, 10.0, 100.0, 1000.0):
149     neighbor_checks[f"linear_minus_{delta:g}"] = total_mean_power(
150         final_linear - delta, final_rotational, params
151     )
152     if final_linear + delta <= 100000.0:
153         neighbor_checks[f"linear_plus_{delta:g}"] = total_mean_power(
154             final_linear + delta, final_rotational, params
155         )
156     neighbor_checks[f"rotational_minus_{delta:g}"] = total_mean_power(
157         final_linear, final_rotational - delta, params
158     )
159
160 checks = {
161     "scan_heave_peak_near_final": abs(scan_damping[np.argmax(heave_scan)] - final_linear)
162     <= 10.0,
163     "scan_pitch_peak_at_upper_boundary": scan_damping[np.argmax(pitch_scan)] == 100000.0,
164     "bounded_heave_matches_analytic": abs(bounded_heave.damping - final_linear) < 1e-2,
165     "bounded_pitch_at_upper_boundary": bounded_pitch.damping == 100000.0,
166     "joint_power_matches_separable_global": abs(-joint.fun - final_total) / final_total <

```

```

1e-9,
166     "joint_linear_near_final": abs(joint.x[0] - final_linear) < 0.1,
167     "joint_rotational_at_boundary": abs(joint.x[1] - final_rotational) < 0.1,
168     "all_neighbor_powers_lower": all(value < final_total for value in
neighbor_checks.values()),
169     "phasor_residuals_small": max(analytic_heave.dynamic_residual,
analytic_pitch.dynamic_residual) < 1e-8,
170     "periodic_state_closure": periodic_closure < 1e-9,
171     "time_frequency_power_agreement": abs(time_total_pto - final_total) / final_total <
1e-9,
172     "time_heave_power_agreement": abs(mean_power_components[2] -
analytic_heave.mean_power) / analytic_heave.mean_power < 1e-9,
173     "time_pitch_power_agreement": abs(mean_power_components[4] -
analytic_pitch.mean_power) / analytic_pitch.mean_power < 1e-9,
174     "periodic_mechanical_energy_closure": abs(mechanical_change) < 1e-7,
175     "periodic_energy_balance": abs(energy_average_residual) < 1e-8,
176     "instantaneous_pto_nonnegative": bool(np.min(powers[[2, 4]]) >= 0.0),
177     "spring_length_positive": float(np.min(spring_length)) > 0.0,
178     "waterline_in_cylinder": float(np.min(draft)) > 0.0 and float(np.max(draft)) <
params.float_cylinder_height,
179     "small_angle_range": maximum_pitch < 0.2,
180 }
181 checks = {name: bool(passed) for name, passed in checks.items()}
182 failed = [name for name, passed in checks.items() if not passed]
183
184 surface_damping = np.linspace(0.0, 100000.0, 201)
185 surface_heave = np.array(
186     [axis_mean_power("heave", value, params).mean_power for value in surface_damping]
187 )
188 surface_pitch = np.array(
189     [axis_mean_power("pitch", value, params).mean_power for value in surface_damping]
190 )
191 np.savez_compressed(
192     models_dir / "q4_power_surface.npz",
193     linear_damping=surface_damping,
194     rotational_damping=surface_damping,
195     total_power=surface_pitch[:, None] + surface_heave[None, :],
196 )
197 np.savez_compressed(
198     models_dir / "q4_optimal_period.npz",
199     time=time,
200     states=solution.y.T,
201     powers=powers.T,
202     state_order=np.array(
203         [
204             "float_heave_displacement",
205             "float_heave_velocity",

```

```

206         "oscillator_heave_displacement",
207         "oscillator_heave_velocity",
208         "float_pitch_displacement",
209         "float_pitch_velocity",
210         "oscillator_pitch_displacement",
211         "oscillator_pitch_velocity",
212     ]
213 ),
214 power_order=np.array(
215     [
216         "wave_input",
217         "heave_radiation",
218         "linear_pto",
219         "pitch_radiation",
220         "rotational_pto",
221     ]
222 ),
223 )
224
225 report: dict[str, object] = {
226     "status": "passed" if not failed else "failed",
227     "model": "CUMCM 2022 A, question 4",
228     "parameter_case": 4,
229     "shell_convention": params.shell_convention.value,
230     "wave_period_s": params.period,
231     "optimization_bounds": [0.0, 100000.0],
232     "analytic_unconstrained_damping": {
233         "linear_N_s_per_m": unconstrained_optimal_damping("heave", params),
234         "rotational_N_m_s": unconstrained_optimal_damping("pitch", params),
235     },
236     "final_result": {
237         "linear_damping_N_s_per_m": final_linear,
238         "rotational_damping_N_m_s": final_rotational,
239         "heave_mean_power_W": analytic_heave.mean_power,
240         "pitch_mean_power_W": analytic_pitch.mean_power,
241         "maximum_total_mean_power_W": final_total,
242         "total_energy_per_period_J": final_total * params.period,
243     },
244     "heave": result_payload(analytic_heave),
245     "pitch": result_payload(analytic_pitch),
246     "bounded_optimizer": {
247         "heave": result_payload(bounded_heave),
248         "pitch": result_payload(bounded_pitch),
249     },
250     "joint_differential_evolution": {
251         "linear_damping": float(joint.x[0]),
252         "rotational_damping": float(joint.x[1]),

```

```

253     "mean_power_W": float(-joint.fun),
254     "evaluations": int(joint.nfev),
255 },
256 "periodic_time_domain_validation": {
257     "sample_count": int(time.size),
258     "state_closure_max_absolute": periodic_closure,
259     "mean_power_components_W": mean_power_components.tolist(),
260     "total_pto_mean_power_W": time_total_pto,
261     "mechanical_energy_change_J": float(mechanical_change),
262     "mean_energy_balance_residual_W": energy_average_residual,
263     "minimum_spring_length_m": float(np.min(spring_length)),
264     "maximum_spring_length_m": float(np.max(spring_length)),
265     "minimum_cylinder_draft_m": float(np.min(draft)),
266     "maximum_cylinder_draft_m": float(np.max(draft)),
267     "maximum_absolute_pitch_rad": maximum_pitch,
268     "maximum_instantaneous_linear_pto_power_W": float(np.max(powers[2])),
269     "maximum_instantaneous_rotational_pto_power_W": float(np.max(powers[4])),
270 },
271 "neighbor_total_powers_W": neighbor_checks,
272 "checks": checks,
273 "passed_count": sum(checks.values()),
274 "total_count": len(checks),
275 "failed": failed,
276 }
277 with (models_dir / "q4_optimization.json").open("w", encoding="utf-8") as stream:
278     json.dump(report, stream, ensure_ascii=False, indent=2)
279 if failed:
280     raise AssertionError("问题四优化验收失败: " + "\n".join(failed))
281 print(json.dumps(report, ensure_ascii=False, indent=2))
282
283
284 if __name__ == "__main__":
285     main()

```